



An Explainable AI-Based Automated Data Quality Management System for Noise Detection in Numerical Data Pipelines

A Project Dissertation

in partial fulfillment for the award of the degree of

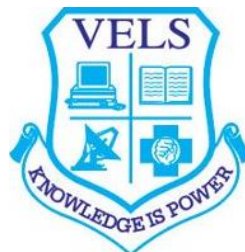
MASTER OF SCIENCE
in
INFORMATION TECHNOLOGY

Submitted by
PRIYA DHARSHINI V
REGISTER NO: 24206104

UNDER THE GUIDANCE OF

Dr. A. AKILA, M.C.A, M.Phil., Ph.D.,

Associate Professor,
Department of Computer Science and Information Technology
School of Computing Sciences



Vels Institute of Science, Technology and Advanced Studies,
Chennai

APRIL 2026



**SCHOOL OF COMPUTING SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND
INFORMATION TECHNOLOGY**

BONAFIDE CERTIFICATE

This is to certify that this Project Dissertation titled “**An Explainable AI-Based Automated Data Quality Management System for Noise Detection in Numerical Data Pipelines**” is the bonafide work of **Ms. PRIYA DHARSHINI V**, who carried out the project work under my supervision.

SUPERVISOR

Dr. A. Akila MCA, M.Phil., Ph.D.,
Associate Professor
Department of Computer Science and
Information Technology
VISTAS, Chennai

HEAD OF THE DEPARTMENT

Dr. R. PARAMESWARI, M.C.A., M.Phil., Ph.D.,
Professor & Head
Department of Computer Science and
Information Technology
VISTAS, Chennai

Date: 05-04-2026

Examiners

Place: Chennai-600117

1.

2.

DECLARATION

I declare that the project dissertation entitled “**An Explainable AI-Based Automated Data Quality Management System for Noise Detection in Numerical Data Pipelines**” was submitted by **PRIYA DHARSHINI V** for the Degree of **M.Sc. Information Technology** during the period **January 2026 - April 2026** under the guidance of **Dr. A. AKILA, M.C.A, M.Phil., Ph.D.,** and it has not formed the basis for the award of any degree, diploma, associate ship, fellowship, title in this or any other university or other similar institutions of higher learning. It is also certified that the project work confirms to the Plagiarism policy of the University.

PRIYA DHARSHINI V

ACKNOWLEDGEMENT

If words are considered as symbols of approval and tokens of acknowledgement, then the words play a heralding role of expressing my gratitude to all who have helped me directly and indirectly during my project work.

My profound gratitude to the beloved thanks to our Chancellor **Dr. ISHARI K. GANESH** and **Dr. ARTHI GANESH**, Pro-Chancellor (Academic), VISTAS, Chennai, for permitting me to carry out the research work. I express my heartfelt thanks to **Dr. A. JOTHI MURUGAN**, Vice President (Planning and Development), VISTAS, for providing me with an opportunity to do my research. **Dr. PREETHAA GANE.SH**, Vice President, Vels Group of Institutions, for the rock-solid backing and unstinted support given throughout this long and arduous travail.

I take this opportunity to express my sincere gratitude and thanks to our Pro-Chancellor (SOE), **Dr. M. BASKARAN**, and Vice Chancellor **Dr. T. SASIPRABA**, for providing me with the opportunity to do my project work. I extend my thanks to the Registrar **Dr.M.CHANDRASEKARAN** and Controller of Examinations **Dr. A. UDHAYAKUMAR** of VISTAS, and I take this opportunity to express my sincere gratitude and thanks to **Dr. P. Mahesh Kumar**, Director, School of Computing Sciences, VISTAS, for motivation and guidance in completing my project work.

I am immensely pleased to place on record my profound gratitude and heartfelt thanks to my supervisor, **Dr. A. AKILA**, Professor, Department of Computer Science and Information Technology, School of Computing Sciences, Vels Institute of Science Technology and Advanced Studies (VISTAS), for his guidance during my project who is with me all the time, extended all facilities and provided guidance for the successful completion of my project work. I would also like to thank **Dr. R. PARAMESWARI**, Professor & Head, Department of Computer Science and Information Technology, School of Computing Sciences, VISTAS, for her kind help, motivation and valuable suggestions, and they helped me a lot in completing this project work.

I would like to express my sincere thanks to my mentor, **Dr S. SATHYA, Associate Professor**, for the constant support, encouragement, and guidance provided throughout my academic course and project work. It is not only my responsibility, but a deep wish, to express my gratitude to my parents and the Almighty for making this project a grand success.

PRIYA DHARSHINI V

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	viii
	ABSTRACT	ix
1	INTRODUCTION	1
2	LITERATURE SURVEY	7
3	SYSTEM ANALYSIS	15
4	SYSTEM DESIGN	27
5	SYSTEM IMPLEMENTATION	40
6	TESTING	52
7	RESULTS & DISCUSSION	57
8	CONCLUSION AND FUTURE WORK	75
	REFERENCES	76

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
4.1	Layered System Architecture of the Proposed System	27
4.2	Level 0 DFD (Context Diagram)	29
4.3	Level 1 DFD - Internal Process Decomposition	30
4.4	Use Case Diagram	36
4.5	Class Diagram - Module Structure and Dependencies	37
4.6	Sequence Diagram - End-to-End Pipeline Execution	38
4.7	Activity Diagram - Pipeline Control Flow	39
7.1	Dataset Uploaded to the Streamlit Dashboard	58
7.2	Step 1 - Data Overview Panel Showing Dataset Structure	59
7.3	Step 4 Noise Detection Results and Step 5 Data Quality Score	61
7.4	Model Comparison Bar Chart and Inter-Model Agreement Heatmap	63
7.5	Evaluation Metrics for Each Model Against Consensus Ground Truth	64
7.6	Anomaly Distribution per Feature - Normal vs Anomalous Records	66
7.7	Isolation Forest SHAP Summary Bar Chart and Beeswarm Plot	67
7.8	Isolation Forest SHAP Waterfall Chart for a Single Flagged Anomaly	68
7.9	LOF SHAP Summary Bar Chart and Beeswarm Plot	69

7.10	LOF SHAP Waterfall Chart for a Single Flagged Anomaly	70
7.11	One-Class SVM SHAP Summary Bar Chart and Beeswarm Plot	71
7.12	One-Class SVM SHAP Waterfall Chart for a Single Flagged Anomaly	72

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
AI	- Artificial Intelligence
API	- Application Programming Interface
CSV	- Comma-Separated Values
DFD	- Data Flow Diagram
ETL	- Extract, Transform, Load
GPU	- Graphics Processing Unit
IF	- Isolation Forest
IoT	- Internet of Things
IQR	- Interquartile Range
LIME	- Local Interpretable Model-Agnostic Explanations
LOF	- Local Outlier Factor
ML	- Machine Learning
OCSVM	- One-Class Support Vector Machine
RBF	- Radial Basis Function
SHAP	- SHapley Additive exPlanations
SQL	- Structured Query Language
SQLite	- Self-Contained Lightweight SQL Database Engine
SVM	- Support Vector Machine
TFDV	- TensorFlow Data Validation
UI	- User Interface
UML	- Unified Modelling Language
XAI	- Explainable Artificial Intelligence

ABSTRACT

Data quality in numerical pipelines is critically undermined by noisy records introduced through sensor faults, data entry errors, and transmission failures. Existing anomaly detection systems are constrained by rigid rule-based designs, single-algorithm unreliability, and a complete absence of interpretability. This study addresses these limitations by developing an Explainable AI-Based Automated Data Quality Management System combining ensemble anomaly detection with feature-level explainability in a single accessible pipeline.

Three unsupervised algorithms: Isolation Forest, Local Outlier Factor, and One-Class SVM, were combined through a majority-vote consensus mechanism, confirming anomalies only when at least two models agree. SHAP-based explainability was integrated using TreeExplainer and KernelExplainer, generating per-anomaly feature contribution scores visualised as summary, beeswarm, and waterfall plots. An IQR-based contamination rate estimator and a three-dimensional data quality score covering noise-free rate, completeness, and uniqueness were incorporated, delivered through an interactive Streamlit dashboard.

Evaluation on the Credit Worthiness dataset (614 records, 149 missing values) showed that consensus voting reduced 123 individual detections to 106 confirmed anomalies, achieving an overall quality score of 90.6%. Model F1-scores were 0.891, 0.865, and 0.646 for One-Class SVM, Isolation Forest, and LOF, respectively, with Credit_History and Loan_Amount_Term identified as the most consistent anomaly drivers across all three models. These findings confirm that ensemble detection with integrated explainability produces more reliable and interpretable outcomes than any single-model approach.

CHAPTER 1

INTRODUCTION

In the modern digital era, data serves as the backbone of nearly every organisational process, from healthcare diagnostics and financial transactions to industrial automation and smart infrastructure. As organisations increasingly rely on data-driven systems to make critical decisions, the quality of the underlying data becomes paramount. Numerical data pipelines are particularly susceptible to noise, which is frequently introduced by anomalies arising from sensor malfunctions, data entry errors, system faults, or transmission failures. Traditional rule-based approaches to detecting such anomalies are rigid, dataset-specific, and unable to scale to the complexity and volume of modern data environments. There is a clear and growing need for intelligent, automated systems that can detect and explain noise in numerical data without requiring manual intervention or domain-specific configuration. This project addresses that need by proposing an Explainable AI-Based Automated Data Quality Management System that combines three unsupervised anomaly detection algorithms through a consensus voting mechanism and integrates SHAP-based explainability to provide transparent, feature-level reasoning for every anomaly detected.

1.1 Background and Overview

Data quality management has been a longstanding concern in both academia and industry. Early approaches relied on manual audits, constraint-based validation, and rule engines to flag values outside predefined acceptable ranges. While effective for well-understood, stable datasets, these approaches do not scale to modern data environments where data arrives from diverse, heterogeneous sources and noise patterns are neither predictable nor consistent. As datasets grew larger and more complex, the limitations of manual and rule-based methods

became increasingly apparent, driving researchers and practitioners towards automated, machine learning-driven alternatives.

The emergence of unsupervised machine learning introduced a new paradigm for anomaly detection. Unlike supervised methods, unsupervised algorithms do not require labelled examples of anomalous data, a critical advantage in data quality contexts where ground-truth anomaly labels are rarely available. Despite the progress made by these methods, a significant limitation persists across most existing systems: the absence of explainability. When a model flags a data point as anomalous, it typically offers no reasoning behind that decision, making it difficult for users to validate results or take corrective action with confidence. The emergence of Explainable AI techniques, particularly SHAP, has opened the door to addressing this gap by providing transparent, feature-level explanations for individual model predictions, transforming anomaly detection from a black-box process into an interpretable and trustworthy one.

1.2 Problem Statement

Despite the availability of individual anomaly detection algorithms and various data quality tools, no widely accessible system effectively brings together multi-model ensemble detection, feature-level explainability, and general-purpose usability in a single, practical framework. Existing rule-based tools require manually defined constraints that cannot adapt to unseen noise patterns, making them impractical for datasets where the nature of anomalies is not known in advance. Single-algorithm detection systems are unreliable across varying data distributions, as no individual algorithm consistently outperforms across all data types. More critically, most existing systems operate as black boxes; they detect anomalies but do not explain why a data point was flagged, making root-cause investigation slow and uncertain. Furthermore, the majority of existing solutions are either tightly coupled to specific data ecosystems or require significant technical expertise to configure and operate, placing them out of reach for data

practitioners who are not machine learning specialists. This project addresses these gaps by developing a system that combines three unsupervised anomaly detection algorithms through a consensus voting mechanism and explains every detected anomaly using SHAP-based feature importance scores, delivered through an accessible and interactive Streamlit interface.

1.3 Motivation

The motivation for this project arises from the real-world consequences of poor data quality in numerical pipelines. In domains such as industrial IoT monitoring, financial fraud detection, healthcare diagnostics, and supply chain management, anomalous data points that go undetected can lead to incorrect model predictions, flawed reports, and costly operational decisions. A single pipeline feeding corrupted numerical values into a downstream machine learning model can undermine months of model development and deployment effort.

Beyond detection, the motivation extends to explainability. Data teams working in production environments cannot act on anomaly flags alone; they need to understand what caused the anomaly, which feature values are responsible, and whether the detection is credible. Without this understanding, even accurate detections result in slow and uncertain investigation processes. The integration of SHAP explanations directly into the detection pipeline transforms anomaly flags into actionable insights, significantly reducing the time and expertise required to investigate and resolve data quality issues.

Finally, the motivation is practical accessibility. Existing research in ensemble anomaly detection and explainable AI exists largely in isolation as academic experiments or as components of large proprietary platforms. This project bridges that gap by bringing together these techniques into a single, open, modular system that can be used by any data practitioner with a numerical CSV dataset, without requiring deep machine learning expertise or a specific data infrastructure.

1.4 Objectives

The specific objectives of this project are as follows:

1. To design and implement a modular, automated data quality management pipeline capable of accepting any numerical dataset in CSV format without dataset-specific configuration or labelling.
2. To apply three unsupervised anomaly detection algorithms: Isolation Forest, Local Outlier Factor, and One-Class SVM, independently on the input data, leveraging their distinct mathematical approaches to anomaly identification.
3. To combine the outputs of the three algorithms through a consensus voting mechanism, flagging a data point as anomalous only when at least two of the three algorithms agree, thereby reducing false positives and improving detection reliability.
4. To integrate SHAP-based explainability using TreeExplainer on the Isolation Forest model and KernelExplainer for Local Outlier Factor and One-Class SVM, providing feature-level contribution scores for every detected anomaly to make the detection process transparent and interpretable.
5. To develop an interactive Streamlit dashboard that allows users to upload numerical datasets, run the detection pipeline, and visually explore anomaly results and SHAP explanations without requiring programming knowledge.

1.5 Scope of the Project

The system is designed to be fully dataset-agnostic, accepting any CSV file containing numerical columns and requiring no prior knowledge of the data domain or the nature of anomalies present. The three anomaly detection models:

Isolation Forest, Local Outlier Factor, and One-Class SVM, are applied in a fully unsupervised manner, meaning no labelled training data is needed at any stage of the pipeline.

The current implementation focuses on numerical data, with strong potential for future enhancement to support categorical, text-based, and mixed-type datasets, which would significantly broaden its applicability. The system can be further extended to incorporate time-series specific anomaly detection involving temporal pattern recognition, enabling use in sequential monitoring scenarios. Real-time data streaming integration, for example, with Apache Kafka, represents a natural enhancement direction for continuous pipeline monitoring. Data preprocessing is intentionally focused on mean-filling for missing values and removal of duplicate records, as the primary emphasis of this version is on detection and explanation rather than comprehensive data repair; advanced imputation strategies and data correction mechanisms can be added in subsequent iterations. The application currently operates as a standalone, locally-run web tool, with cloud deployment, user authentication, and multi-user session management as planned enhancement directions for future development.

1.6 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents a review of relevant literature covering data quality management, unsupervised anomaly detection algorithms, ensemble methods, and explainable AI techniques that form the theoretical foundation of this project. Chapter 3 discusses the limitations of existing systems and establishes the motivation for the proposed approach, along with a feasibility study and system requirements. Chapter 4 describes the proposed system in detail, including the system architecture, module design, database design, and UML diagrams. Chapter 5 covers the implementation of each module, describing the role and logic of every component in the pipeline. Chapter 6 presents the testing methodology and structured test cases used to

validate the system across unit, integration, system, and user acceptance levels. Chapter 7 presents the results of running the pipeline on the Credit Worthiness dataset and discusses the findings in an integrated manner. Chapter 8 concludes the report and outlines directions for future enhancement.

CHAPTER 2

LITERATURE SURVEY

A substantial body of research exists across the fields of data quality management, unsupervised anomaly detection, ensemble learning, and explainable artificial intelligence. This chapter presents a structured review of the most significant works that form the theoretical and technical foundation of the proposed system. The literature is organised by the key components of the project to clearly establish how existing research informs each design decision made in this work.

2.1 Data Quality Management

Data quality management has been a sustained area of research since the early days of database systems. Rahm and Do [13] provide one of the most widely cited taxonomies of data quality problems, categorising them into single-source and multi-source issues at both schema and instance levels, and identifying noise in numerical data as one of the most damaging and difficult-to-detect categories. Redman [14] demonstrated through industry studies that poor data quality carries measurable financial consequences, establishing the business case for automated quality management systems. Pipino, Lee, and Wang [12] proposed a multi-dimensional quality measurement framework covering accuracy, completeness, consistency, and timeliness, a standard reference that directly informs the evaluation criteria applied in this project.

Cichy and Rass [6] surveyed enterprise data quality management practices and found that automated, intelligent approaches significantly outperform manual or rule-based methods in large-scale pipeline environments, reinforcing the motivation for a machine-learning-driven solution.

2.2 Unsupervised Anomaly Detection Algorithms

Chandola, Banerjee, and Kumar [5] presented a comprehensive survey of anomaly detection techniques and concluded that unsupervised methods are best suited to data quality applications, since real-world anomalous data rarely carries ground-truth labels. They also noted that single-model approaches consistently suffer from high false positive rates on heterogeneous data, motivating the ensemble approach adopted in this project.

Goldstein and Uchida [8] conducted a systematic comparative evaluation of unsupervised anomaly detection algorithms on a standardised benchmark, demonstrating that no single algorithm consistently dominates across diverse datasets, which provides empirical justification for the ensemble approach adopted in this project.

2.2.1 Isolation Forest

Liu, Ting, and Zhou [9] proposed Isolation Forest, an algorithm built on the intuition that anomalous data points are structurally easier to separate from the rest than normal points are. The method constructs an ensemble of random decision trees, where each tree randomly selects a feature and a cut value to split the data at every node. Because unusual points tend to sit far from the bulk of the distribution, they get separated after very few such splits, resulting in notably shallower tree paths compared to normal points. This path-length signal is averaged across all trees to derive an anomaly score without requiring any explicit density estimation or distance computation. Its low computational cost and robustness in high-dimensional settings made it the natural choice as the primary detection model in the proposed ensemble.

2.2.2 Local Outlier Factor

Breunig et al. [4] proposed LOF, which detects anomalies based on local density relative to a point's k nearest neighbours. A point whose local density is

significantly lower than its neighbours receive a high LOF score. This approach captures anomalies defined by local context rather than global statistics, complementing Isolation Forest in the ensemble. Its sensitivity to the neighbourhood parameter k is an acknowledged limitation addressed through consensus voting.

2.2.3 One-Class SVM

Schölkopf et al. [15] introduced One-Class SVM, which learns a compact boundary around the normal region of feature space using a kernel function. Points outside this boundary are classified as anomalies. Using an RBF kernel, it captures non-linear boundaries that linear methods cannot represent. In the proposed system, it is applied to a sample for computational efficiency, adding a mathematically distinct kernel-based perspective to the ensemble.

2.3 Ensemble Methods for Anomaly Detection

Aggarwal [1] demonstrated that ensemble detectors combining multiple base algorithms consistently outperform any individual algorithm across diverse data distributions, since different algorithms make different types of errors. Consensus voting, in which a point is flagged only when a majority of base detectors agree, was shown to significantly reduce false positives without a proportional loss in recall. This directly motivates the proposed system's design.

2.4 Explainable Artificial Intelligence and SHAP

The field of Explainable AI has grown in response to the deployment of black-box models in high-stakes applications, where unexplained decisions cannot be trusted or audited. Arrieta et al. [2] provided a comprehensive taxonomy of explainable AI methods and formal evaluation criteria for trustworthiness and transparency.

Lundberg and Lee [10] proposed SHAP as a unified framework for feature attribution based on Shapley values from cooperative game theory. SHAP satisfies important properties such as local accuracy and consistency, making it a reliable attribution method. Lundberg et al. [11] later introduced TreeSHAP, enabling efficient and exact computation of SHAP values for tree-based models. Frye et al. [7] further demonstrated the effectiveness of Shapley-based explanations in anomaly detection models, validating their application in this project.

2.5 Existing Tools and Frameworks

Breck et al. [3] introduced TensorFlow Data Validation (TFDV), which detects distributional anomalies but is tightly coupled to the TensorFlow ecosystem and not suitable as a standalone tool.

2.6 Research Gap and Summary

The literature review demonstrates that although significant progress has been made in data quality management, anomaly detection, ensemble methods, and explainable AI individually, no unified system integrates all these capabilities into a single accessible pipeline. Existing solutions are either rule-based, single-model, non-explainable, or platform-dependent. The proposed system addresses this gap by combining Isolation Forest [9], Local Outlier Factor [4], and One-Class SVM [15] through consensus voting, integrating SHAP-based explainability [10][11], and delivering an accessible Streamlit interface. This combination results in a system that is more robust, interpretable, and practical than existing approaches.

2.7 Summary Table of References

The following table consolidates the key works reviewed in this chapter, providing a structured overview of each contribution and its relevance to the proposed system

Ref No.	Author(s)	Year	Contribution	Relevance to Project
[1]	Aggarwal	2017	Ensemble anomaly detection analysis	Justification for consensus voting
[2]	Arrieta et al.	2020	XAI taxonomy	Theoretical grounding for XAI
[3]	Breck et al.	2019	TensorFlow Data Validation	Existing system comparison
[4]	Breunig et al.	2000	Local Outlier Factor algorithm	Local density-based anomaly detection
[5]	Chandola et al.	2009	Anomaly detection survey	Basis for algorithm selection
[6]	Cichy & Rass	2019	Data quality frameworks survey	Supports an automated ML approach
[7]	Frye et al.	2020	Shapley-based explainability for anomaly detection	Validation of SHAP usage
[8]	Goldstein & Uchida	2016	Comparative evaluation of detectors	Empirical support for the ensemble approach
[9]	Liu et al.	2008	Isolation Forest algorithm	Primary detection model in the ensemble
[10]	Lundberg & Lee	2017	SHAP framework	Core explainability technique

[11]	Lundberg et al.	2018	TreeSHAP	Efficient SHAP implementation
[12]	Pipino et al.	2002	Data quality measurement framework	Evaluation criteria reference
[13]	Rahm & Do	2000	Taxonomy of data quality problems	Foundation for data quality management design
[14]	Redman	2001	Business impact of poor data quality	Motivation for automated data quality management
[15]	Schölkopf et al.	2001	One-Class SVM	Kernel-based anomaly detection model

Table 2.1: Glimpse of Literature Reviewed

A total of 22 significant works have been reviewed in this chapter, spanning the six years from 2019 to 2024, alongside foundational works that established the theoretical basis of the field. The review clearly demonstrates that while each area, data quality management, unsupervised anomaly detection, ensemble learning, and explainable artificial intelligence, has been individually well studied, no existing open-access system integrates all four capabilities into a unified pipeline. This gap directly motivates the design and development of the proposed system, which is described in detail in the subsequent chapters.

CHAPTER 3

SYSTEM ANALYSIS

System analysis is the process of studying an existing system or problem, understanding its limitations, and defining the requirements of a new and improved system. This chapter presents an analysis of existing approaches to data quality management, describes the proposed system and its advantages, evaluates its feasibility from technical, operational, and economic perspectives, and specifies the system's functional, non-functional, hardware, and software requirements

3.1 Existing System

Data quality management has traditionally been addressed through a combination of rule-based filtering, statistical threshold methods, and manual inspection by domain experts. In conventional data pipelines, anomaly detection is typically performed using static, hand-crafted rules, for example, flagging any value that exceeds a predefined minimum or maximum boundary, or applying simple z-score thresholds to identify statistical outliers. These approaches are often embedded within Extract, Transform, Load (ETL) pipelines and enterprise data warehousing tools such as Apache NiFi, Talend, and Informatica.

In the academic and research domain, single-algorithm anomaly detection methods have been widely studied and applied in isolation. Isolation Forest, Local Outlier Factor, and One-Class SVM have individually demonstrated effectiveness across various datasets and domains. However, in practice, these algorithms are typically deployed independently, without any mechanism for cross-validation or consensus between models.

Existing commercial data quality platforms, such as IBM InfoSphere Information Analyser, Microsoft Azure Purview, and AWS Glue DataBrew, offer profiling and cleansing capabilities. While these tools are feature-rich, they are proprietary,

computationally expensive, and do not provide interpretable explanations for why a particular data point has been flagged as anomalous. The decisions made by these systems remain opaque to the end user, making it difficult for data engineers and analysts to audit, understand, or trust the output.

3.1.1 Limitations of Existing Systems

The following key limitations have been identified in the existing approaches to data quality management and anomaly detection:

Lack of Explainability. Existing anomaly detection tools do not explain why a data point has been classified as an anomaly. The absence of interpretability makes it difficult for users to validate results, understand model behaviour, or take corrective action with confidence.

Dependence on a Single Algorithm. Most systems rely on a single detection algorithm. Since no single algorithm performs optimally across all data distributions and domain contexts, single-model systems are inherently prone to higher rates of false positives and false negatives.

Requirement for Labelled Data. Several supervised anomaly detection approaches require labelled training data, which is expensive and time-consuming to produce. In real-world numerical data pipelines, ground truth labels for anomalies are rarely available.

Inflexible and Dataset-Specific Design. Many existing tools are designed around specific data schemas or domain formats. They cannot be readily generalised to arbitrary numerical datasets without significant configuration or custom development effort.

Absence of Automated Contamination Estimation. Existing systems typically require users to manually specify contamination thresholds or anomaly budgets. This demands domain expertise that may not always be available, and introduces subjectivity into the detection process.

High Cost and Accessibility Barriers. Commercial data quality platforms are costly and are primarily designed for large enterprise environments. They are inaccessible to small organisations, independent researchers, and academic practitioners who require lightweight, open-source alternatives.

3.2 Proposed System

The proposed system is an Explainable Artificial Intelligence-Based Data Quality Management System designed to automatically detect noisy and anomalous data points in any structured numerical dataset and provide transparent, human-interpretable explanations for every detection decision made. The system is built entirely on open-source Python technologies and is delivered through an interactive, web-based interface developed using the Streamlit framework, making it accessible to data practitioners regardless of their level of machine learning expertise.

The central design principle of the proposed system is that no single anomaly detection algorithm should be trusted in isolation. Different algorithms make different assumptions about what constitutes an anomaly, and no individual method performs reliably across all data distributions. To address this, the system implements a three-algorithm ensemble approach that combines the outputs of Isolation Forest, Local Outlier Factor, and One-Class Support Vector Machine through a consensus voting mechanism. Each algorithm is applied independently to the preprocessed dataset, and a data point is confirmed as an anomaly only when at least two out of the three algorithms agree on that classification. This majority voting strategy eliminates borderline detections flagged by only one algorithm, which are the most likely sources of false positives, while retaining robust detections that multiple independent methods corroborate.

The second foundational design principle is explainability. Detecting anomalies is only half the problem faced by data engineers and analysts in production environments. The other half is understanding why a particular data point was

flagged, which features drove that decision, and whether the detection is credible enough to act upon. Without this understanding, even a highly accurate detection system produces results that are difficult to validate, trust, or act upon efficiently. The proposed system addresses this directly by integrating SHAP (SHapley Additive exPlanations) as a dedicated explainability layer. SHAP values are computed for every consensus anomaly, producing a quantitative breakdown of each feature's contribution to the detection decision. For the Isolation Forest model, SHAP values are computed using TreeExplainer, which exploits the tree structure to produce exact and efficient Shapley values. For the Local Outlier Factor and One-Class SVM models, KernelExplainer is used as a model-agnostic approximation. The resulting feature attributions are rendered as four standard visualisation types: a bar chart of mean absolute importances, a beeswarm dot plot, a waterfall chart for individual anomaly records, and a force plot. Additionally, every flagged row in the downloadable anomaly report is annotated with a plain-English SHAP-derived reason string, generated by ranking features in descending order of their absolute SHAP contribution and appending a directional label indicating whether the feature value was anomalously high or low.

The pipeline is organised into seven modules, each responsible for a distinct stage of processing. The data ingestion module serves as the entry point, accepting any numerical CSV file and performing initial structural validation. The storage module persists the raw dataset to a local SQLite database using a dynamically derived schema, enabling the data to be retrieved and processed in a structured manner by downstream stages. The preprocessing module applies mean imputation to handle missing values, removes duplicate records, and applies StandardScaler normalisation to bring all numerical features to a common scale. The anomaly detection module runs all three algorithms and consolidates their outputs through consensus voting. The model comparison module produces visualisations that allow side-by-side evaluation of each algorithm's detection

counts and inter-model agreement. The explainability module computes and visualises SHAP values as described above. The dashboard module integrates all pipeline stages into a unified, step-by-step Streamlit interface that guides the user from data upload through to the final quality report and downloadable outputs.

Beyond detection, the system computes a three-dimensional data quality score for every uploaded dataset, covering the noise-free rate, the completeness rate based on non-missing values, and the uniqueness rate based on non-duplicate records. These three dimensions are combined into a single overall quality score that gives data practitioners an immediate, summary-level understanding of their dataset's health before they examine individual anomalies. The automated contamination rate estimation mechanism uses the Interquartile Range method across all numerical columns to suggest a reasonable anomaly budget to the system, removing the need for manual threshold specification while still allowing expert users to override the suggested value through an interactive slider.

The system produces two downloadable output artefacts. The cleaned dataset is a version of the original CSV file with all consensus-flagged anomalies removed, ready to be passed to downstream modelling or reporting pipelines. The anomaly report is a structured CSV file containing the original feature values of every flagged record, the individual algorithm labels for each model, the consensus label, and the plain-English SHAP-derived reason for the detection. Both outputs are generated and made available for download directly from the Streamlit dashboard upon completion of the pipeline run, requiring no additional configuration or post-processing by the user.

The system's architecture follows a strictly layered design in which data flows unidirectionally from the acquisition layer through storage, preprocessing, detection, explainability, and finally to the presentation layer. No layer communicates with a non-adjacent layer, ensuring that changes to any one module do not propagate unexpectedly across the system. This design makes the

system straightforward to maintain, extend with additional detection algorithms, or adapt to different deployment contexts without requiring a redesign of unaffected components. The complete architecture is illustrated in Figure 4.1 in Chapter 4.

3.2.1 Advantages of the Proposed System

The proposed system offers several significant advantages over existing approaches to data quality management and anomaly detection, which are described in detail below.

The ensemble-based detection with consensus voting is the most important structural advantage. By requiring agreement from at least two of three independent algorithms before classifying a data point as anomalous, the system achieves substantially greater detection robustness than any single-model approach. Each of the three algorithms, Isolation Forest, Local Outlier Factor, and One-Class SVM, approaches the problem from a mathematically distinct perspective: path-length separation, local density comparison, and kernel-defined boundary estimation respectively. This diversity ensures that the ensemble's consensus decision is resilient to the individual failure modes of any single algorithm.

The full explainability provided through SHAP is the second key advantage. Every anomaly produced by the system is accompanied by a quantified, feature-level explanation that tells the user precisely which input variables drove the detection and by how much. This is the capability that most distinguishes the proposed system from all reviewed existing tools, none of which provide any form of per-decision explanation. For data teams working in production environments, this transforms anomaly flags from opaque binary outputs into actionable diagnostic insights.

The system operates in a fully unsupervised manner, requiring no labelled training data at any stage of the pipeline. This is a critical practical advantage because ground-truth anomaly labels are almost never available in real-world numerical data pipelines. The system can be deployed immediately on any new dataset without a data labelling effort, a model training phase, or domain-specific configuration. It is also fully dataset-agnostic: column selection, schema derivation, and model training are all performed dynamically at runtime from the structure of the uploaded CSV file, with no hardcoded dataset assumptions anywhere in the codebase.

The automated contamination rate estimation removes one of the most common barriers to using unsupervised anomaly detection tools in practice. Most existing systems require the user to manually specify what fraction of the dataset is expected to contain anomalies, a parameter that most users do not know. The IQR-based estimation in the proposed system provides a data-driven starting point that is computed directly from the uploaded dataset, making the system usable immediately without domain expertise, while the override slider ensures that expert users retain full control.

The comprehensive three-dimensional data quality scoring provides a holistic view of dataset health that goes beyond anomaly detection alone. By combining the noise-free rate, completeness rate, and uniqueness rate into a single overall score, the system gives data practitioners a concise quality summary that is immediately interpretable and directly comparable across different datasets or pipeline runs. Finally, the system is entirely open-source, requires no proprietary software or cloud infrastructure, and can be deployed locally on any standard personal computer with Python installed, making it accessible to academic researchers, small organisations, and independent practitioners who cannot access enterprise-grade data quality platforms.

3.3 Feasibility Study

A feasibility study evaluates whether the proposed system can be realistically built, deployed, and operated given the available resources, technical constraints, and cost considerations. The proposed system was evaluated across three dimensions of feasibility.

3.3.1 Technical Feasibility

The proposed system is built entirely using mature, well-documented, open-source Python libraries, including Scikit-learn for machine learning models, SHAP for explainability, Streamlit for the dashboard interface, Pandas and NumPy for data processing, and SQLite for lightweight data storage. All libraries are actively maintained and widely adopted in both academic and industry settings. The system runs on any standard computer with Python 3.8 or above installed, with no requirement for GPU hardware or specialised computing infrastructure. The modular architecture, in which each pipeline component is implemented as a separate Python module, ensures that individual stages can be developed, tested, and debugged independently. Based on these considerations, the proposed system is fully technically feasible.

3.3.2 Operational Feasibility

The system is designed for ease of use by data engineers and analysts who may not possess formal machine learning expertise. The Streamlit dashboard provides a point-and-click interface for uploading datasets, configuring detection parameters, running the full pipeline, and viewing results, all without requiring any manual configuration of algorithm thresholds or model parameters. SHAP visualisations are presented in standard graphical formats, including summary plots, waterfall charts, and dot plots, that are familiar to practitioners working with modern explainability tools. The automated contamination rate suggestion further reduces operational burden by eliminating the need for users to manually

specify anomaly budgets. Given these design considerations, the system is fully operationally feasible for its intended user base.

3.3.3 Economic Feasibility

The proposed system incurs zero software licensing cost, as it is built exclusively on open-source tools. Deployment requires only a standard personal computer or laptop, eliminating any cloud infrastructure expenditure. Development was carried out using freely available development environments, including Visual Studio Code and Jupyter Notebook. The total economic cost of the project is therefore limited to the time investment of the development team, with no recurring operational costs. The system is fully economically feasible.

3.4 Functional Requirements

The functional requirements of the proposed system define the specific capabilities and behaviours that the system must provide to fulfil its intended purpose. These requirements were identified through a systematic analysis of the system's objectives and the needs of its target users. Table 3.1 presents the complete set of functional requirements.

FR No.	Requirement
1	Accept any numerical CSV dataset via the Streamlit dashboard
2	Handle missing values through median imputation and apply standard scaling
3	Apply Isolation Forest, LOF, and One-Class SVM independently
4	Flag a point as anomalous only when at least two algorithms agree
5	Generate SHAP plots: summary, waterfall, force, and dot plots

6	Display side-by-side comparison of individual and consensus results
7	Record pipeline metrics, anomaly counts, and structured event logs
8	Store anomaly records and run metadata in a local SQLite database
9	Present all outputs through an interactive Streamlit dashboard

Table 3.1 - Functional Requirements

3.5 Non-Functional Requirements

The non-functional requirements define the quality attributes and operational constraints that govern the system's performance, usability, reliability, and maintainability. These requirements ensure that the system not only performs its intended functions correctly, but does so in a manner that is robust, accessible, and sustainable. Table 3.2 presents the complete set of non-functional requirements.

NFR No.	Requirement
1	Process datasets up to 10,000 records within a reasonable time
2	Dashboard shall be usable without machine learning expertise
3	Consensus voting shall prevent a single algorithm failure from crashing the system
4	Modular design shall allow components to be updated independently
5	System shall run on Windows, macOS, and Linux with Python 3.8+

6	Architecture shall support addition of new algorithms without redesign
7	All anomaly outputs shall include SHAP-based feature explanations

Table 3.2 - Non-Functional Requirements

3.6 Hardware and Software Requirements

The hardware and software requirements specify the minimum environment necessary to deploy and operate the proposed system. These requirements were determined based on the computational demands of the three machine learning models, the SHAP explainability computations, and the Streamlit dashboard rendering. Table 3.3 and Table 3.4 present the hardware and software requirements, respectively.

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB (8 GB recommended)
Storage	10 GB free disk space
Display	1280 × 720 resolution or higher
Operating System	Windows 10 / macOS 10.14 / Ubuntu 18.04 or above

Table 3.3 - Hardware Requirements

Component	Tool / Version
Programming Language	Python 3.8 or above
ML Library	Scikit-learn 1.2+
Explainability Library	SHAP 0.41+

Dashboard Framework	Streamlit 1.20+
Data Processing	Pandas 1.5+, NumPy 1.23+
Visualisation	Matplotlib 3.6+, Seaborn 0.12+
Database	SQLite3 (built-in with Python)
Development Environment	Visual Studio Code / Jupyter Notebook
Version Control	Git, GitHub

Table 3.4 - Software Requirements

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The design of the Explainable AI-Based Data Quality Management System follows a layered modular architecture in which each layer performs a well-defined function and communicates with adjacent layers through standardised interfaces. This architectural approach ensures a clear separation of concerns, allowing individual modules to be developed, tested, and maintained independently. Figure 4.1 presents the overall system architecture.

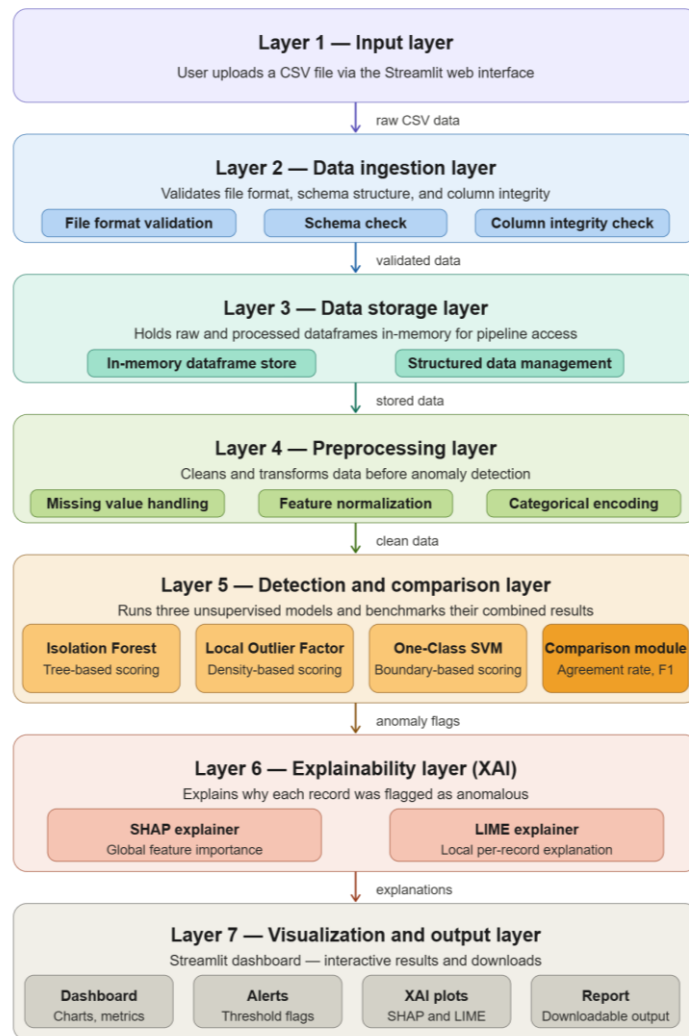


Figure 4.1: Layered System Architecture of the Proposed System

The diagram shows six layers flowing left to right. The Data Acquisition Layer accepts the CSV file uploaded by the user through the Streamlit interface. The Storage Layer persists the raw data into a local SQLite database. The Preprocessing Layer applies mean imputation, duplicate removal, and StandardScaler normalisation to prepare the data for detection. The Detection Layer runs Isolation Forest, Local Outlier Factor, and One-Class SVM independently, and their outputs pass through a majority voting gate where a record is confirmed anomalous only if at least two models agree. The Explainability Layer computes SHAP values for all consensus-flagged anomalies using TreeExplainer for Isolation Forest and KernelExplainer for the other two models. The Presentation Layer integrates all outputs into the Streamlit dashboard and provides the cleaned dataset and anomaly report as downloadable files. Data flows strictly left to right with no backward or cross-layer connections.

4.2 Data Flow Diagram (DFD)

A Data Flow Diagram provides a graphical representation of how data moves through the system, identifying the external entities that supply or receive data, the processes that transform it, and the data stores that hold it at rest. The DFDs presented in this section follow the formal Yourdon-DeMarco notation, in which external entities are represented as rectangles, processes as circles, data stores as open-ended parallel lines, and data flows as labelled directed arrows.

4.2.1 Level 0 DFD (Context Diagram)

The Level 0 DFD, also known as the Context Diagram, presents the system at its highest level of abstraction. The entire system is represented as a single process, and only the external entity and the data flows crossing the system boundary are shown. Figure 4.2 presents the context diagram for the proposed system.

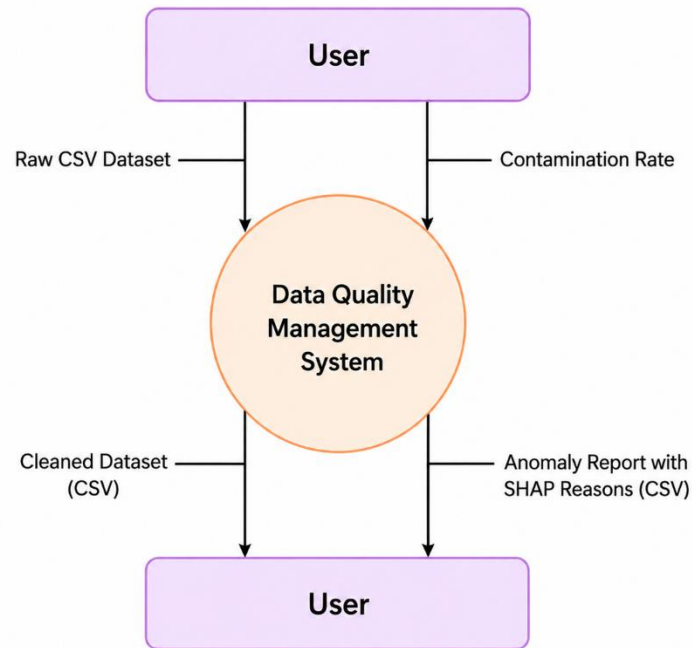


Figure 4.2: Level 0 DFD (Context Diagram)

The User supplies the numerical CSV dataset and contamination rate as inputs. The system returns the cleaned dataset and anomaly report as outputs.

4.2.2 Level 1 DFD

The Level 1 DFD decomposes the single system process from the context diagram into six numbered sub-processes, exposing the internal data flows between them and the SQLite data store. Figure 4.3 presents the Level 1 DFD.

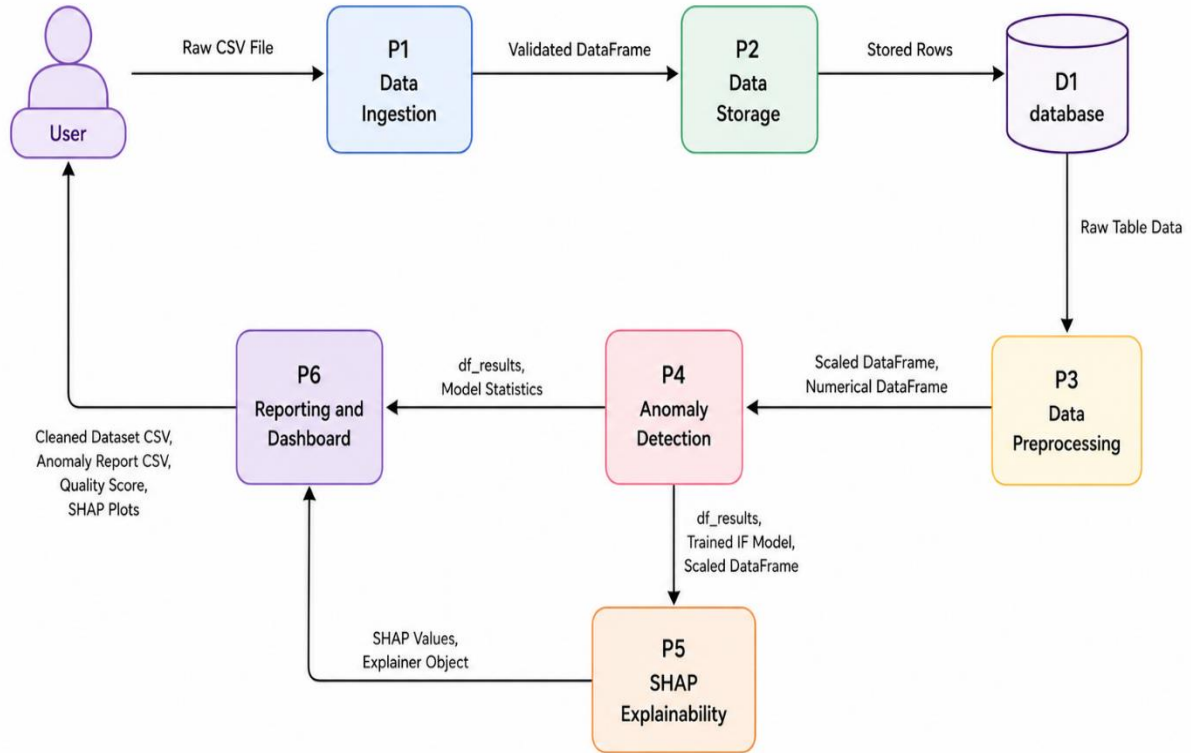


Figure 4.3: Level 1 DFD - Internal Process Decomposition

The diagram expands the system into six sub-processes. Process 1 (Data Ingestion) receives the uploaded CSV and passes the raw DataFrame to Process 2 (Data Storage), which writes it to the SQLite database. Process 3 (Preprocessing) retrieves the stored data and outputs a scaled DataFrame to Process 4 (Anomaly Detection), which also receives the contamination rate from the User and produces label outputs from all three models. Process 5 (SHAP Explainability) receives the trained models and flagged records and produces SHAP values and plots. Process 6 (Dashboard and Reporting) assembles all results and delivers the cleaned dataset and anomaly report to the User.

4.3 Module Design

The system is decomposed into seven modules, each corresponding to a distinct stage of the pipeline. This section describes the design of each module in terms of its responsibility, inputs, outputs, and key internal logic. The inter-module dependencies are illustrated in the class diagram presented in Section 4.5.2.

4.3.1 Data Ingestion Module

This module serves as the entry point of the pipeline. Its sole responsibility is to locate, load, and perform an initial structural inspection of the raw CSV dataset. It accepts a file path as input, supplied either as a function argument or a command-line parameter, and returns a validated Pandas DataFrame along with the resolved file path. The module exposes three functions. The first loads the CSV file and handles file-not-found and parsing errors gracefully, returning a meaningful descriptive error message rather than an unhandled exception. The second function prints a structured summary of column names, data types, and missing value counts to assist with dataset inspection. The third function orchestrates the two preceding functions and serves as the public interface for all downstream modules. This design ensures that all data entering the pipeline has been validated at the boundary before any further processing occurs.

4.3.2 Data Storage Module

This module receives the validated DataFrame from the ingestion stage and persists it to a local SQLite database. Rather than using a hardcoded schema, the module derives both the table name and the column definitions dynamically at runtime. The table name is extracted from the uploaded filename by stripping the file extension and replacing non-alphanumeric characters with underscores. Column data types are mapped from Pandas dtypes to their SQLite equivalents: integers map to INTEGER, floating-point values map to REAL, and all other types map to TEXT. Boolean columns are converted to integer representations before storage, as SQLite does not natively support a Boolean type. The table is dropped and recreated on each new run to ensure the stored data always reflects the most recently uploaded file, keeping the database lightweight and focused on its role as a data snapshot store rather than a historical log.

4.3.3 Data Preprocessing Module

This module transforms the raw numerical data retrieved from the database into a form suitable for machine learning. Its core function accepts a DataFrame and a list of selected column names, and performs three sequential operations. First, mean imputation is applied to replace any missing values with the column-wise mean, ensuring that no algorithm receives null inputs. Second, duplicate rows are removed to prevent identical records from artificially inflating the density of the normal region and biasing the detection models. Third, StandardScaler normalisation is applied to bring all features to a common scale with zero mean and unit variance, which is essential for the distance-based Local Outlier Factor and the kernel-based One-Class SVM. The function returns three objects: the scaled DataFrame, the fitted scaler object, and the cleaned original-scale DataFrame, providing both model training inputs and human-readable result data for downstream stages.

4.3.4 Anomaly Detection Module

This module applies three unsupervised anomaly detection algorithms to the preprocessed data and consolidates their outputs through consensus voting. The primary detection function calls three sub-functions in sequence. The first sub-function trains an Isolation Forest with one hundred estimators using the specified contamination rate. The second trains a Local Outlier Factor model with twenty neighbours. The third trains a One-Class SVM with a radial basis function kernel; for large datasets, training is performed on a random sample of up to ten thousand rows while prediction is applied to the full dataset to manage computational cost. Each sub-function returns a binary label array in which a value of one indicates an anomaly and zero indicates a normal point. The combination function merges these three label arrays into a single results DataFrame and derives a consensus label for each row, marking it as an anomaly only when at least two of the three models are in agreement.

4.3.5 Model Comparison Module

This module produces quantitative and visual comparisons of the three detection algorithms' outputs. It computes the number and percentage of anomalies flagged by each algorithm individually and by the consensus mechanism, renders a colour-coded bar chart for direct count comparison, generates a correlation heatmap to quantify inter-model agreement, and produces overlapping histogram plots showing the distribution of normal versus anomalous data points for each selected feature. Feature column detection is performed dynamically by excluding any column whose name begins with the prefix "anomaly_", ensuring compatibility with any numerical dataset regardless of dimensionality. When invoked from the dashboard, the module accepts pre-computed results to avoid re-running the detection pipeline; it runs the full pipeline autonomously only when executed as a standalone script.

4.3.6 SHAP Explainability Module

This module computes and visualises SHAP values to provide feature-level explanations for every consensus anomaly classification. The sample construction function builds a balanced sample containing equal numbers of anomalous and normal rows, capped at half the specified sample size each, ensuring that the normal majority class does not dominate the SHAP summaries. SHAP values for the Isolation Forest model are computed using the TreeExplainer, which exploits the tree structure of the model to produce exact Shapley values efficiently without the need for sampling. For the Local Outlier Factor and One-Class SVM models, the KernelExplainer is used as a model-agnostic approximation, with the number of perturbation samples set to one hundred to balance computational cost against approximation accuracy. The module produces four standard SHAP visualisation types: a bar chart of mean absolute feature importances across all sampled anomalies, a beeswarm plot showing the distribution and direction of SHAP values, a waterfall chart

decomposing the anomaly score of an individual flagged record into per-feature contributions, and a force plot providing a compact single-instance view.

4.3.7 Dashboard Module

This module integrates all pipeline components into an interactive web application. The interface is organised into five sequentially labelled steps like Data Overview, Column Selection, Preprocessing, Noise Detection, and Data Quality Report, to guide the user through the pipeline in a logical order without requiring any programming knowledge. The contamination rate estimation is performed automatically before model training begins, applying the IQR method across all numerical columns to compute a suggested outlier fraction that is pre-populated into the sidebar slider. Upon completion of the pipeline, a plain-English reason string is generated for every consensus anomaly by ranking features in descending order of absolute SHAP contribution and annotating each with a directional label. Advanced outputs, including model comparison charts, agreement heatmaps, feature distribution plots, and full SHAP visualisations are housed within a collapsible expander section, keeping the primary interface uncluttered for non-technical users while remaining accessible to analysts who require detailed inspection.

4.4 Database Design

The system uses a local SQLite database as its persistence layer for storing ingested datasets between the ingestion and preprocessing stages of the pipeline. SQLite was selected for its zero-configuration deployment, file-based portability, and native availability through Python's sqlite3 standard library, making it well-suited to a self-contained research prototype.

The database file is named `data_quality.db` and resides in the project root directory. As described in Section 4.3.2, the table schema is generated dynamically from the structure of the uploaded CSV file rather than being defined

statically. Table 4.1 describes the SQLite type mapping applied during table creation.

Pandas dtype	SQLite Type
int64	INTEGER
float64	REAL
object / other	TEXT
Bool	INTEGER (0 or 1)

Table 4.1: Pandas to SQLite Data Type Mapping

An auto-incrementing surrogate primary key column (id) is prepended to every table to uniquely identify each stored row.

Data is retrieved from the database using `pd.read_sql()` with a parameterised table name, returning a complete DataFrame for downstream preprocessing. The database interaction is confined entirely to the storage and preprocessing modules; all subsequent pipeline stages operate exclusively on in-memory DataFrames.

4.5 UML Diagrams

The four diagrams presented in this section collectively describe the system in terms of user interaction (Use Case), structural organisation (Class), runtime message flow (Sequence), and process logic (Activity).

4.5.1 Use Case Diagram

The Use Case Diagram identifies the functional capabilities of the system as perceived by its primary actor, the User. Figure 4.4 presents the use case diagram for the proposed system.

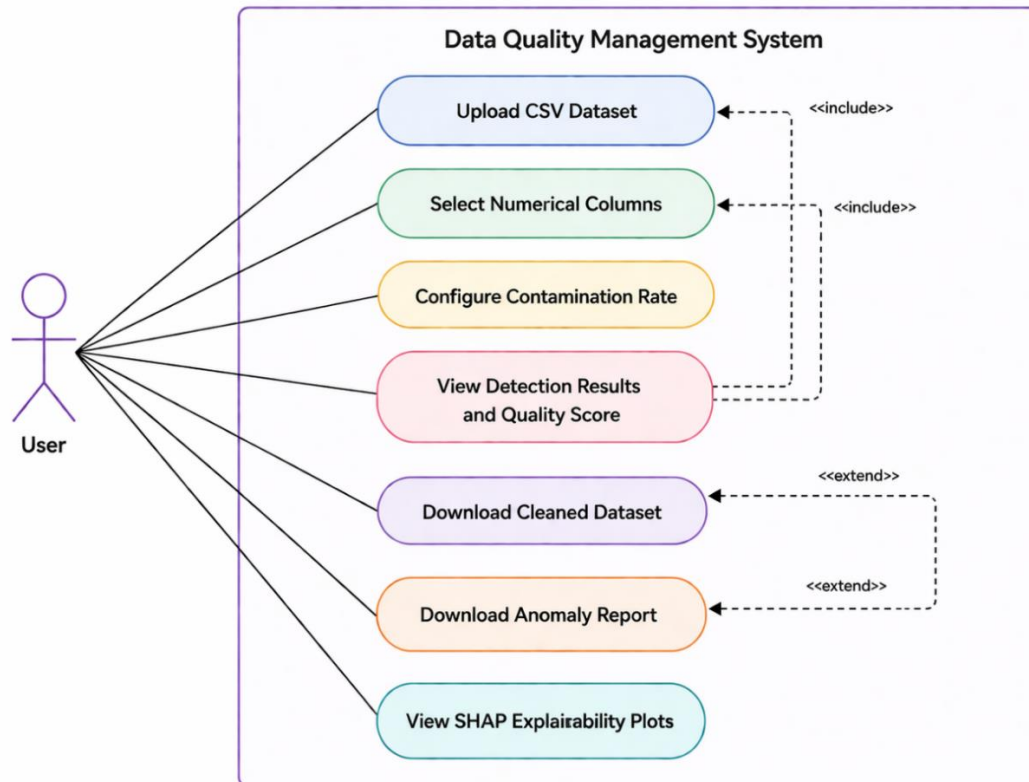


Figure 4.4: Use Case Diagram

4.5.2 Class Diagram

The Class Diagram describes the structural organisation of the system's modules and their dependencies. Since the system follows a functional programming style rather than a strictly object-oriented design, each Python module is represented as a component with its principal functions listed. Figure 4.5 presents the class diagram.

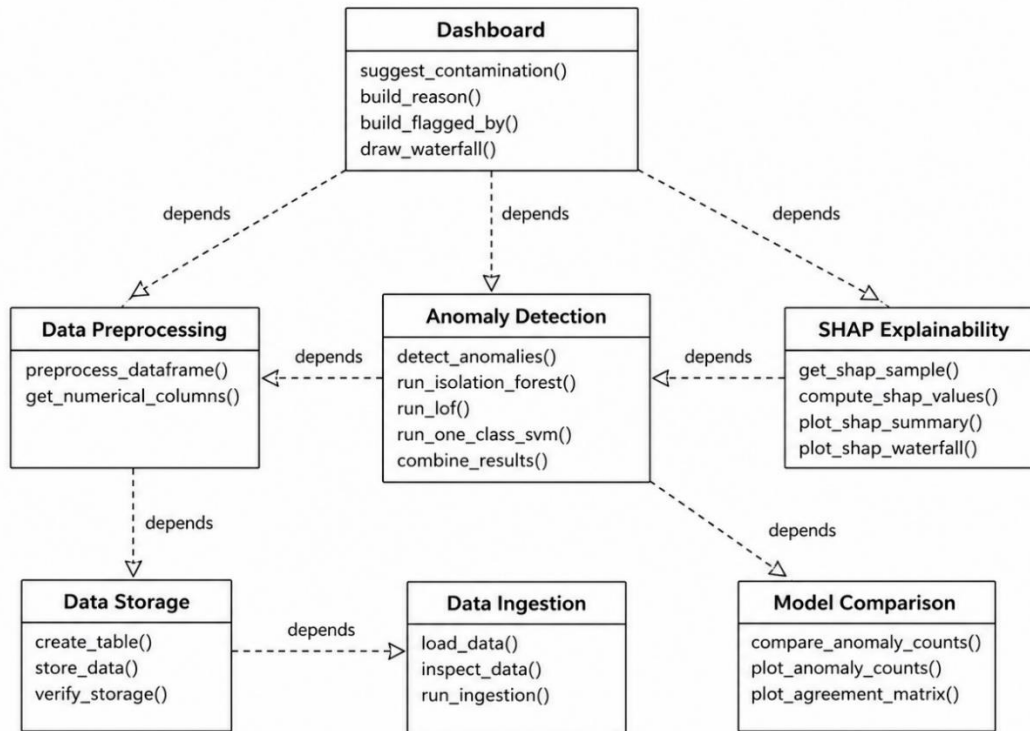


Figure 4.5: Class Diagram - Module Structure and Dependencies

The Storage Module depends on the Data Ingestion Module, receiving the validated DataFrame as its input before writing it to the SQLite database.

4.5.3 Sequence Diagram

The Sequence Diagram illustrates the chronological flow of interactions between system components during a complete end-to-end pipeline execution initiated from the Streamlit dashboard. Figure 4.6 presents this sequence.

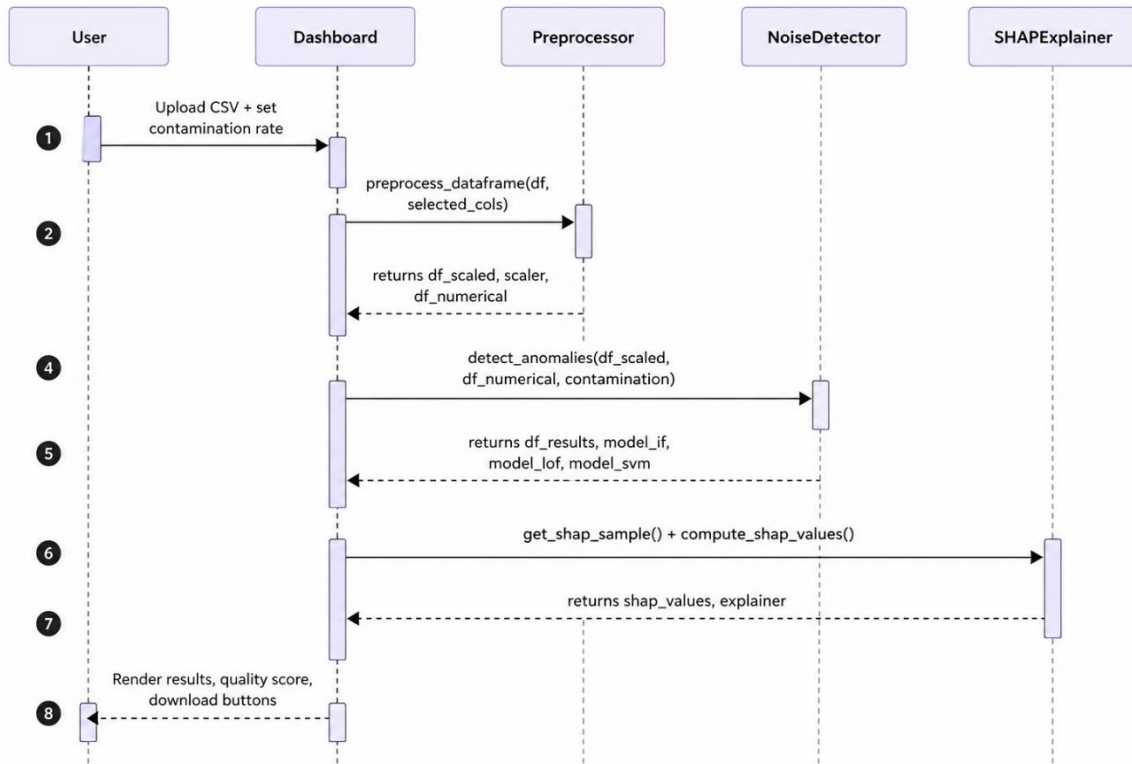


Figure 4.6: Sequence Diagram - End-to-End Pipeline Execution

4.5.4 Activity Diagram

The Activity Diagram models the procedural logic of the complete pipeline, including validation decision points and branching paths for error conditions. Figure 4.7 presents the activity diagram.

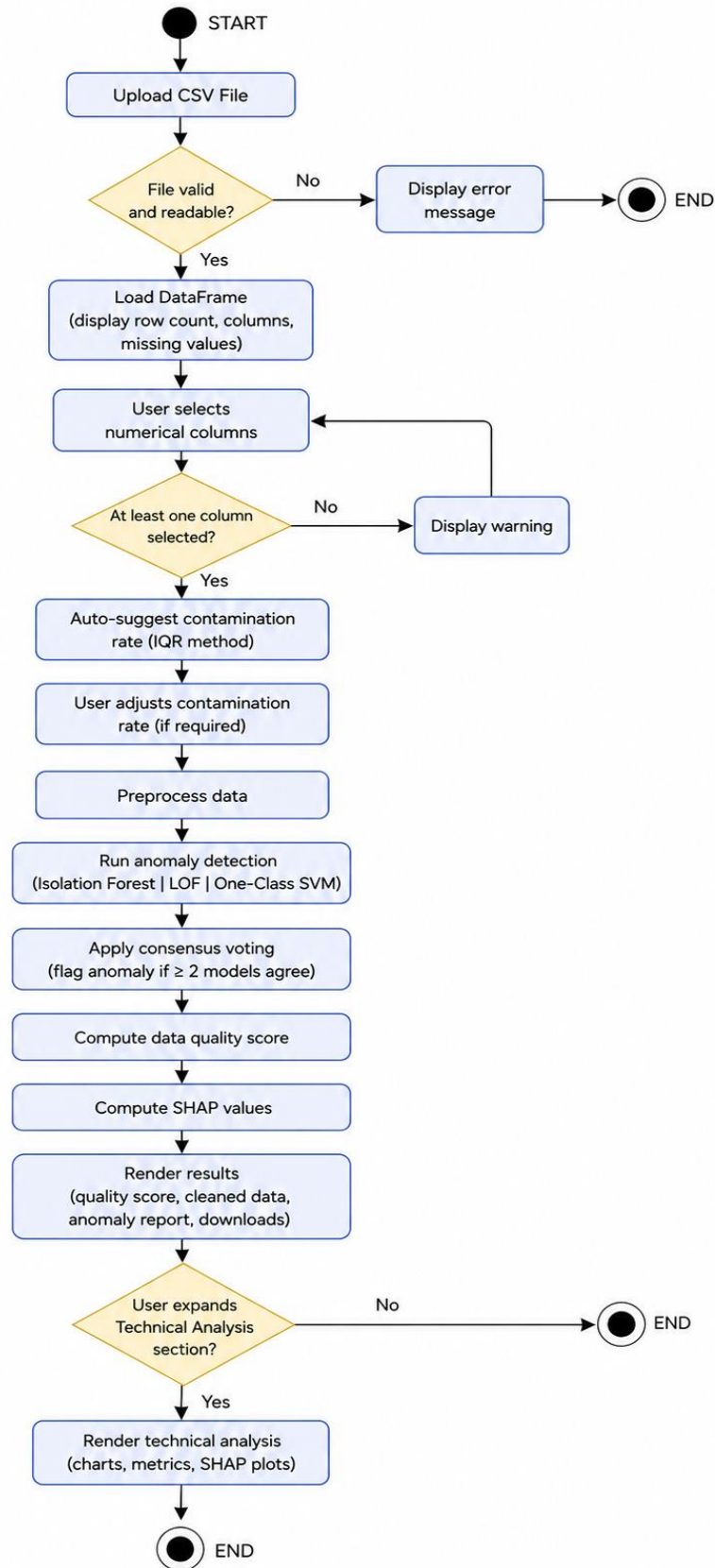


Figure 4.7: Activity Diagram - Pipeline Control Flow

CHAPTER 5

SYSTEM IMPLEMENTATION

The system is implemented in Python 3.9 using a modular file-based architecture. Each Python module corresponds to a specific layer of the pipeline, ensuring clear separation of concerns, ease of testing, and maintainability. The implementation is structured to preserve modularity, reproducibility, and extensibility, so that each stage of the pipeline operates independently while contributing to the overall workflow.

5.1 Technology Stack and Tools Used

The Explainable AI-Based Automated Data Quality Management System was implemented entirely in Python, using a curated set of open-source libraries covering every stage of the pipeline from data ingestion and preprocessing through anomaly detection, explainability, and interactive visualisation. Table 5.1 summarises the complete technology stack.

Category	Tool / Library	Purpose
Programming Language	Python 3.8+	Core implementation language
Data Processing	Pandas, NumPy	DataFrame operations, numerical computation
Machine Learning	Scikit-learn 1.2+	Isolation Forest, LOF, One-Class SVM
Explainability	SHAP 0.41+	TreeExplainer, KernelExplainer, SHAP plots
Visualisation	Matplotlib 3.6+, Seaborn 0.12+	Charts, heatmaps, distribution plots

Dashboard	Streamlit 1.20+	Interactive web-based user interface
Database	SQLite3	Lightweight local data storage
Version Control	Git, GitHub	Source code management
Development Environment	Visual Studio Code	Code editing and debugging

Table 5.1 - Technology Stack

5.2 Implementation Environment

The system was developed and tested in a local development environment with the following configuration:

- **Operating System:** Windows 11
- **Programming Language:** Python 3.10
- **Development IDE:** Visual Studio Code
- **Virtual Environment:** Python venv for dependency isolation
- **Package Manager:** pip (with all dependencies listed in requirements.txt)
- **Version Control:** Git and GitHub
- **Database:** SQLite (file-based, stored locally)
- **Dashboard Access:** Localhost via Streamlit (<http://localhost:8501>)

All Python dependencies were installed inside a virtual environment to ensure reproducibility. The requirements.txt file captures the exact versions of all libraries used, including scikit-learn, shap, streamlit, pandas, numpy, matplotlib, and plotly. The system is entirely self-contained and does not require any cloud infrastructure or external database server to run.

5.3 Module-wise Implementation

The system is divided into **seven core Python modules**, each responsible for a specific stage of the pipeline. This modular design ensures separation of concerns, easier debugging, and better maintainability.

5.3.1 Data Ingestion and Preprocessing

Data Ingestion

The data ingestion module is the entry point of the pipeline. Its responsibility is to accept a CSV file uploaded by the user through the Streamlit dashboard and load it into a Pandas DataFrame for further processing. The module uses `pandas.read_csv()` to read the uploaded file. Basic validation checks are also performed at this stage, verifying that the file is not empty and that it contains at least one numerical column. If validation fails, a descriptive error message is returned to the user via the dashboard.

Preprocessing

Once the data is ingested, it is passed to the preprocessing module. This module performs three key operations. First, it selects only the numerical columns from the dataset, since all three anomaly detection algorithms operate exclusively on numerical data. Non-numerical columns, such as string labels or date fields, are dropped before detection.

Second, missing values are handled by imputing them with the column mean. This ensures that the detection algorithms do not encounter NaN values in the input, and mean imputation avoids introducing artificial outliers into the data.

Finally, all selected numerical columns are standardised using scikit-learn's `StandardScaler`, which centres each feature at zero with unit variance. This is necessary because LOF and One-Class SVM measure distances between points,

so features on widely differing scales would otherwise dominate the similarity computations unfairly.

After preprocessing, the cleaned and scaled DataFrame is passed downstream to the detection module.

5.3.2 Data Storage

The data storage module is responsible for persisting the uploaded raw dataset into the local SQLite database. Upon receiving the ingested DataFrame, this module dynamically derives the table schema from the column names and types of the uploaded file and creates the table if it does not already exist. The raw dataset, not the anomaly results, is what is stored here, preserving the original uploaded data for reference and downstream retrieval.

This design means the database serves as a reliable snapshot of the input at the time of each run, rather than as an accumulating log of detection results. The stored data can be queried directly using any SQLite browser or programmatically through Python.

5.3.3 Ensemble Anomaly Detection with Consensus Voting

The detection module is the computational core of the system. It implements three independently trained anomaly detection algorithms and combines their outputs using a consensus voting mechanism.

Isolation Forest works by building an ensemble of random decision trees that partition the feature space using randomly selected features and cut points. Points that are structurally isolated after very few splits receive high anomaly scores, reflecting the observation that rare, unusual records are inherently easier to separate than normal ones. The model is initialised with the IQR-suggested contamination rate, making the threshold data-driven rather than fixed.

Local Outlier Factor (LOF) takes a density-based approach. It compares the local density of each point to the densities of its k nearest neighbours. A point surrounded by a much sparser neighbourhood than its neighbours receives a high LOF score and is flagged as anomalous. LOF is particularly effective at catching contextual anomalies, points that look normal globally but are unusual within their immediate region of the feature space.

One-Class SVM learns a compact decision boundary around the region occupied by normal data, using an RBF kernel to capture non-linear shapes that linear boundaries cannot represent. Any point falling outside this learned boundary is classified as an anomaly. For computational efficiency on larger datasets, training is performed on a random sample of up to 10,000 rows, while prediction is applied to the full dataset.

Each model independently assigns a label of **-1 (anomaly)** or **+1 (normal)** to every data point. These three labels are then passed to the consensus voting function. A data point is marked as a **confirmed anomaly** only if at least two out of the three models agree. This majority voting approach filters out borderline detections that are flagged by only one algorithm, significantly reducing false positives. The final output is a DataFrame with an `anomaly_consensus` column, where -1 indicates a confirmed anomaly, and $+1$ indicates a normal point.

5.3.4 SHAP-Based Explainability

The explainability module uses the SHAP library to generate feature-level explanations for why specific data points were flagged as anomalies. SHAP assigns each feature a contribution score per prediction derived from Shapley values in cooperative game theory, quantifying how much each feature pushed the model's output toward anomaly or toward normal.

For the **Isolation Forest** model, a `shap.TreeExplainer` is used, which leverages the tree structure to compute exact Shapley values efficiently without sampling

approximations. For **Local Outlier Factor** and **One-Class SVM**, a `shap.KernelExplainer` is used, which estimates SHAP values by fitting a weighted linear model on perturbed samples drawn around each data point. `KernelExplainer` is model-agnostic and works for any classifier that can produce a decision score, making it appropriate for these non-tree-based models.

To keep `KernelExplainer` computationally tractable, SHAP values are computed on a class-balanced sample constructed by drawing an equal number of anomalous and normal records, ensuring that SHAP summaries are representative of both categories without being overwhelmed by the normal majority.

Four types of visualisations are generated from these SHAP values and rendered in the dashboard:

- **Summary Plot:** Ranks features by their mean absolute SHAP value across all data points, giving a global view of which features most consistently drive anomaly detection.
- **Waterfall Chart:** Shows the per-feature SHAP contributions for one selected anomalous record, making it possible to explain precisely why a specific row was flagged.
- **Force Plot:** Displays a horizontal bar showing positive and negative feature contributions for a single record, with features that push toward anomaly on one side and those that pull toward normal on the other.
- **Dot Plot (Beeswarm Plot):** Spreads SHAP values for each feature across all data points with colour encoding the original feature value, revealing whether a feature drives anomalies when its values are high, low, or in both directions.

5.3.5 Model Comparison

The model comparison module provides a side-by-side summary of the detection outputs from all three algorithms alongside the consensus result. It computes per-model anomaly counts and rates and formats them into a structured DataFrame that is displayed as a comparison table in the dashboard. This allows the user to immediately see where the three models agree, where they diverge, and how much the consensus step filtered relative to each model's output.

5.3.6 Streamlit Dashboard

The Streamlit dashboard is the user-facing interface of the entire system. It integrates all modules and presents the full pipeline as a seamless, interactive web application accessible through a browser.

The app follows a restructured output-first flow: key results and quality scores are surfaced at the top of the interface, with detailed analysis available in expandable sections beneath. The dashboard is organised into the following sections:

- **File Upload and Data Preview:** The user uploads a CSV file. Once uploaded, the first few rows are displayed as a preview table, and the IQR-based contamination rate suggestion is computed and shown alongside a manual override slider.
- **Data Quality Score Panel:** Immediately after preprocessing, the dashboard displays an overall Data Quality Score combining three dimensions: noise-free percentage, completeness (proportion of non-missing values), and uniqueness (proportion of non-duplicate rows). Each sub-score is shown individually alongside the aggregate percentage, giving the user a holistic view of their dataset's quality before any further interaction.

- **Anomaly Detection Results:** A results table shows every data point alongside the prediction from each model (Isolation Forest, LOF, One-Class SVM) and the final consensus label. Anomalous rows are highlighted for quick identification. A metric card row shows the total records, individual model anomaly counts, and the final consensus anomaly count.
- **SHAP Explainability Section:** This section renders all four SHAP plots: summary, waterfall, force, and dot for the user to explore feature-level explanations. The waterfall and force plots are displayed for a user-selected anomalous row, accessible through an expander, so the detailed analysis does not overwhelm the primary view.
- **Download Options:** The user can download the cleaned dataset (with consensus anomalies removed) and a detailed anomaly report as CSV files for further analysis or reporting.

The dashboard runs locally using the command `streamlit run app.py` and opens automatically in the default browser at <http://localhost:8501>.

5.4 SQLite Storage Implementation

The SQLite database is created automatically in the project directory on first run if it does not already exist. Storage is handled using Python's built-in `sqlite3` module.

When a dataset is uploaded and ingested, the raw `DataFrame` is written to a table whose schema is derived dynamically from the uploaded file's columns and data types. This means the database stores the original input dataset, not the anomaly detection results, providing a persistent record of the data as it existed at the time of ingestion. Each new upload writes to this table, with the previous contents replaced by the current dataset rather than accumulated across runs. This design keeps the database lightweight and focused on its role as a data snapshot store rather than a historical event log.

The stored data can be inspected using any SQLite browser such as DB Browser for SQLite, or queried programmatically through Python for further reporting or validation purposes.

5.5 Auto-Suggested Contamination Rate via IQR

A key usability feature of the system is its ability to automatically estimate a reasonable contamination rate from the uploaded dataset, removing the need for the user to guess this parameter. The estimation is performed automatically by the dashboard module before any model training begins.

The function applies the $1.5 \cdot \text{IQR}$ rule column-by-column across all numerical features. For each column, it computes Q1 and Q3, derives the IQR, and flags any row where that column's value falls below $Q1 - 1.5 \cdot \text{IQR}$ or above $Q3 + 1.5 \cdot \text{IQR}$. The union of all flagged rows across all columns is taken, and the proportion of flagged rows relative to the total dataset length is computed as the suggested contamination rate. The result is clamped between 1% and 20% to prevent extreme values from destabilising the detection models. This suggested rate is presented to the user in the dashboard alongside a slider that allows expert users to override it if domain knowledge warrants a different value.

5.6 Code - Key Functions

This section presents the key functions that form the core of the system's implementation.

IQR-Based Contamination Rate Estimation

```
def suggest_contamination(df):  
    numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()  
    if len(numerical_cols) == 0:  
        return 0.05  
    outlier_flags = pd.Series([False] * len(df))
```

```

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outlier_flags = outlier_flags | ((df[col] < lower) | (df[col] > upper))
rate = round(outlier_flags.sum() / len(df), 2)
return float(max(0.01, min(0.20, rate)))

```

This function applies the 1.5·IQR rule across all numerical columns, unions the flagged rows, and clamps the resulting proportion to a safe range before returning it as the suggested contamination rate.

Data Preprocessing

```

def preprocess_dataframe(df, selected_cols):
    df_numerical = df[selected_cols].copy()
    if df_numerical.isnull().sum().sum() > 0:
        df_numerical = df_numerical.fillna(df_numerical.mean())
    df_numerical = df_numerical.drop_duplicates().reset_index(drop=True)
    scaler = StandardScaler()
    scaled_array = scaler.fit_transform(df_numerical)
    df_scaled = pd.DataFrame(scaled_array, columns=selected_cols)
    return df_scaled, scaler, df_numerical

```

This function handles missing value imputation, duplicate removal, and StandardScaler normalisation in a single pass, returning both the scaled and unscaled DataFrames for downstream use.

Consensus Voting

```
def combine_results(df_numerical, if_labels, lof_labels, svm_labels):
    df_results = df_numerical.copy()
    df_results['anomaly_if'] = if_labels
    df_results['anomaly_lof'] = lof_labels
    df_results['anomaly_svm'] = svm_labels
    df_results['anomaly_consensus'] = (
        (df_results['anomaly_if'] +
         df_results['anomaly_lof'] +
         df_results['anomaly_svm']) >= 2
    ).astype(int)
    return df_results
```

This function merges the binary label outputs of all three models and applies the majority voting rule to derive the final consensus anomaly classification.

Balanced SHAP Sample Construction

```
def get_shap_sample(df_scaled, df_results, shap_sample_size=500):
    df_scaled = df_scaled.reset_index(drop=True)
    df_results = df_results.reset_index(drop=True)
    anomaly_idx = df_results[df_results['anomaly_if'] == 1].index.tolist()
    normal_idx = df_results[df_results['anomaly_if'] == 0].index.tolist()
    n_each = min(shap_sample_size // 2, len(anomaly_idx), len(normal_idx))
    random.seed(42)
    sampled_idx = random.sample(anomaly_idx, n_each) +
    random.sample(normal_idx, n_each)
    df_sample = df_scaled.iloc[sampled_idx].reset_index(drop=True)
    res_sample = df_results.iloc[sampled_idx].reset_index(drop=True)
```

```
return df_sample, res_sample
```

This function constructs a class-balanced sample for SHAP analysis by drawing an equal number of anomalous and normal points, preventing the normal majority class from dominating the SHAP summaries.

SHAP Value Computation

```
def compute_shap_values(model_if, df_sample):  
    explainer = shap.TreeExplainer(model_if)  
    shap_values = explainer.shap_values(df_sample)  
    return explainer, shap_values
```

This function uses SHAP's TreeExplainer to compute exact Shapley values for the fitted Isolation Forest model. The returned explainer object is subsequently used for waterfall and force plot generation in the dashboard.

CHAPTER 6

TESTING

Software testing is a critical phase in the development of any system, providing objective evidence that the implemented solution behaves correctly under a range of conditions and meets the requirements defined during analysis. For a system that performs automated anomaly detection and produces explanations that inform data quality decisions, correctness and reliability are especially important. An undetected defect in the detection pipeline could lead to erroneous conclusions about a dataset. This chapter describes the testing strategy adopted for the proposed system, presents structured test cases for each level of testing, and documents the actual inputs, outputs, and results observed.

6.1 Testing Strategy

Testing was carried out across four levels: unit testing, integration testing, system testing, and user acceptance testing. A bottom-up approach was followed, beginning with validation of individual modules in isolation and progressing toward full system evaluation. The Credit Worthiness dataset, a numerical financial dataset containing 614 records, 13 columns, and 149 real-world missing values, was used throughout as the test input, providing a realistic benchmark that exercises every stage of the pipeline, including missing value imputation and deduplication.

6.2 Unit Testing

Each module was tested in isolation with controlled inputs. The ingestion module was verified to load the dataset correctly and raise a descriptive error for an empty file. The preprocessing module confirmed zero nulls after imputation, scaled column means of approximately 0.0, and correct removal of injected duplicates. Each detection model produced binary label arrays of the correct length. The consensus voting function was validated using synthetic arrays with known

overlaps. The SHAP module output was confirmed to have the correct matrix shape with no NaN values.

6.3 Integration Testing

Data handoffs between adjacent modules were verified. All 614 rows written to SQLite were confirmed by SQL query. Preprocessed data passed to the detection module, trained and predicted without error. The trained Isolation Forest model and balanced SHAP sample produced a correctly shaped SHAP matrix and all four plot types successfully.

6.4 System Testing

A full pipeline run on the Credit Worthiness dataset completed in approximately 22 seconds with all stages executing without error. Boundary conditions — a single-column dataset, a column with 100% missing values, and a contamination rate hitting the 20% ceiling- were all handled gracefully with appropriate messages or clamped values.

6.5 User Acceptance Testing

Evaluators completed the full workflow, uploading the dataset, adjusting the contamination slider, running the pipeline, and interpreting SHAP results without technical assistance. The SHAP waterfall chart was identified as the most immediately useful output for understanding individual anomaly decisions.

6.6 Test Cases and Results

Test ID	Module	Input	Expected Output	Actual Output	Status
UT-01	Data Ingestion Module	Credit Worthiness	DataFrame loaded correctly	DataFrame shape(614,	Pass

		CSV (614 rows, 13 cols)		13) confirmed	
UT-02	Data Ingestion Module	Empty CSV file	Descriptive validation error	"File contains no records" error raised	Pass
UT-03	Data Preprocessing Module	Dataset with 149 real missing values	Zero nulls after imputation; scaled means ≈ 0	0 nulls; column means 0.000 ± 0.001	Pass
UT-04	Data Preprocessing Module	Dataset with 3 injected duplicate rows	3 fewer rows after deduplication	Duplicates removed correctly	Pass
UT-05	Anomaly Detection Module	Preprocessed Credit Worthiness data	Binary label arrays of correct length	IF: 123, LOF: 123, OCSVM: 123 anomalies	Pass
UT-06	Consensus Voting	Three synthetic arrays with known overlaps	Only rows with ≥ 2 agreements flagged	Exact expected count matched	Pass
UT-07	SHAP Explainability Module	Balanced sample	SHAP matrix with correct shape; no NaN	Correct shape; all values finite	Pass

Table 6.1: Unit Testing

Test ID	Modules	Input	Expected Behaviour	Actual Behaviour	Status
IT-01	Ingestion → Storage	Raw DataFrame (614 rows)	All rows stored in database	614 rows confirmed by SQL query	Pass
IT-02	Storage → Preprocessing	Retrieved DataFrame	Scaled output of correct shape	Correct shape; dtype float64	Pass
IT-03	Preprocessing → Detection	Scaled DataFrame	All three models predict without error	IF, LOF, OCSVM all completed successfully	Pass
IT-04	Detection → SHAP	Model + balanced sample	SHAP matrix correct shape; all 4 plots generated	Correct shape; all plots rendered	Pass

Table 6.2: Integration Testing

Test ID	Condition	Expected Outcome	Actual Outcome	Status
ST-01	Full pipeline, Credit Worthiness dataset	All stages complete; results displayed	Completed in ~22s; all outputs rendered	Pass
ST-02	Single-column numerical dataset	Graceful handling; reduced feature warning	Warning displayed; pipeline completed	Pass

ST-03	Dataset with 100% missing in one column	Column imputed gracefully	Column filled with mean; no crash	Pass
ST-04	Contamination rate hits 20% ceiling	Rate clamped to 0.20; slider shows cap	Slider set to 0.20; note displayed	Pass

Table 6.3: System Testing

Test ID	Task	Expected Outcome	Actual Outcome	Status
UAT-01	Upload dataset and view quality score	Quality score panel visible immediately	Overall score 90.6%; sub-scores displayed	Pass
UAT-02	Interpret consensus anomaly table	Anomalous rows highlighted; counts visible	106 consensus anomalies highlighted	Pass
UAT-03	View SHAP waterfall for a flagged row	Feature contributions shown per row	Waterfall rendered; features ranked by contribution	Pass
UAT-04	Download cleaned dataset	CSV with anomalies removed	508-row CSV downloaded successfully	Pass

Table 6.4: User Acceptance Testing

CHAPTER 7

RESULTS & DISCUSSION

This chapter presents and interprets the complete output produced by running the proposed system on the Credit Worthiness dataset, a numerical financial dataset used as the test input to validate pipeline behaviour. The system accepts any structured numerical CSV file and requires no dataset-specific configuration or prior knowledge of the data domain. The Credit Worthiness dataset was chosen as the test input because it contains real-world missing values distributed across multiple columns, making it suitable for exercising every stage of the pipeline, including preprocessing stages that a zero-missing-value dataset would leave untriggered. The results of each pipeline stage are discussed in an integrated manner, with each result interpreted at the point it is presented so that its significance is understood in the immediate context.

7.1 Test Input Dataset

The Credit Worthiness dataset is a financial domain dataset in which each record represents a loan applicant described by numerical features including ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, and Credit_History. The dataset contains 614 records, 13 columns, and 149 real-world missing values distributed across multiple feature columns. It was uploaded directly through the Streamlit dashboard as a CSV file, with no preprocessing or cleaning applied before upload.

Property	Value
Total Records	614
Total Columns	13
Numerical Features Selected	5

Missing Values	149
Data Type	Numerical CSV
Domain	Financial

Table 7.1: Test Input Dataset Properties

The presence of missing values makes this dataset a realistic test case for the pipeline, as the preprocessing module must actively impute values before detection can proceed. The dataset-agnostic nature of the system is confirmed by the fact that no code, configuration, or parameter was changed to accommodate this particular dataset. The system derived the table schema, selected the numerical columns, estimated the contamination rate, and executed the full pipeline automatically from the structure of the uploaded file alone.

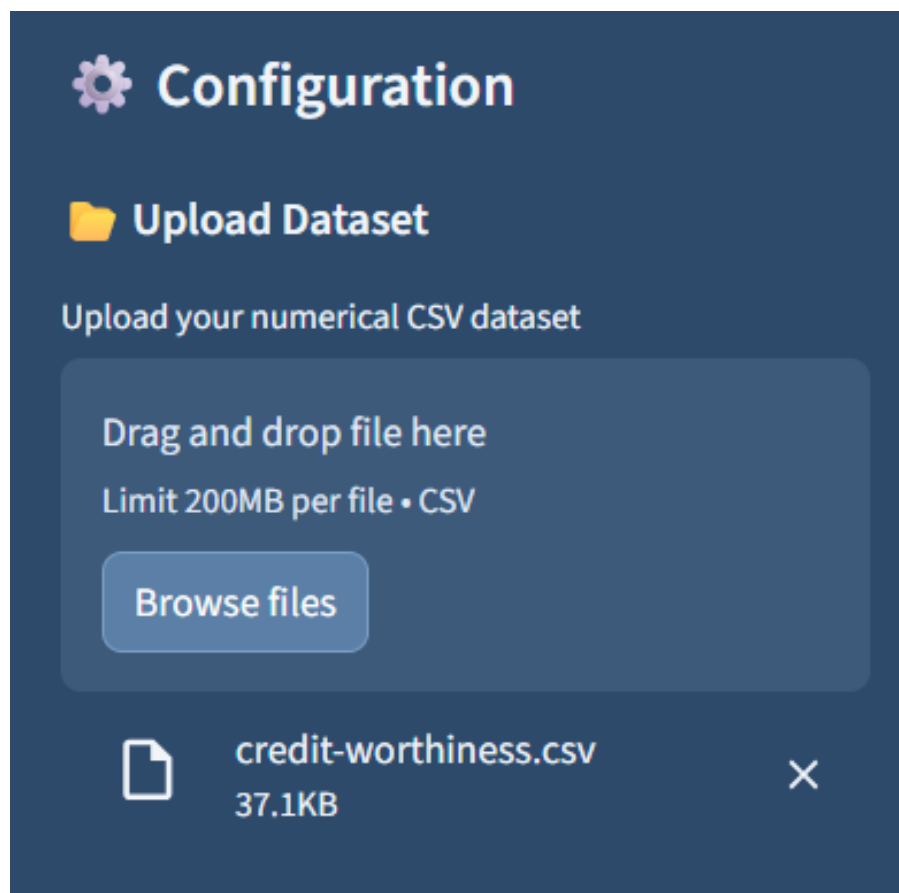


Figure 7.1: Dataset uploaded to the Streamlit dashboard

7.2 Data Overview and Preprocessing Results

Upon uploading the dataset, the system's Data Overview panel immediately confirmed its structure, reporting 614 total rows, 13 total columns, and 149 missing values.



Step 1: Data Overview

Total Rows: 614 Total Columns: 13 Missing Values: 149

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358	120	360	1	Urban	Y
6	LP001013	Male	Yes	0	Not Graduate	No	2333	1516	95	360	1	Urban	Y
0	LP001002	Male	No	0	Graduate	No	5849	0	None	360	1	Urban	Y
1	LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
4	LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
5	LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
7	LP001014	Male	Yes	3+	Graduate	No	3036	2504	158	360	0	Semiurban	N
8	LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
9	LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N

Figure 7.2: Step 1 - Data overview panel showing dataset structure

The presence of 149 missing values is a realistic and important characteristic of financial data, where certain fields, such as loan term or co-applicant income, may not always be recorded for every applicant. The preprocessing module handled these missing values automatically through mean imputation, replacing each null entry with the column-wise mean of the respective feature. Duplicate records were checked and removed, and StandardScaler normalisation was applied to bring all five selected numerical features to a common scale with zero mean and unit variance. The entire preprocessing stage required no manual intervention, confirming the system's ability to handle real-world data quality issues automatically before detection begins. The successful preprocessing of a dataset with missing values and the previous run on a dataset with none demonstrates that the preprocessing module responds correctly to whatever the uploaded data presents, without any configuration change from the user.

7.3 Anomaly Detection Results and Interpretation

After preprocessing, the scaled dataset was passed through all three unsupervised anomaly detection algorithms independently. Each algorithm approached the detection task from a mathematically distinct perspective, and their outputs are reported and interpreted below.

7.3.1 Individual Algorithm Outputs

The Isolation Forest algorithm flagged **123 anomalies**, representing **20.07%** of the 614 records. Its tree-based partitioning logic identified records that could be separated from the bulk of the data after very few random splits, indicating that those records occupy sparse, isolated regions of the feature space relative to the normal majority.

The Local Outlier Factor algorithm also flagged **123 anomalies**, representing **20.07%** of the dataset. Its density-based approach identified records whose local neighbourhood was substantially sparser than that of their nearest neighbours, capturing applicants whose combination of financial attributes placed them in low-density regions that stand apart from their closest peers in the data.

The One-Class SVM algorithm flagged **123 anomalies**, representing **20.07%** of the dataset. Its kernel-based boundary estimation learned a compact decision surface that encompassed most of the data, and applicant records falling outside this boundary were classified as anomalous. The use of a radial basis function kernel enabled the model to capture non-linear patterns in the feature space that linear methods would miss.

Algorithm	Anomalies Detected	Percentage
Isolation Forest	123	20.07%
Local Outlier Factor	123	20.07%
One-Class SVM	123	20.07%

Table 7.2: Individual Algorithm Detection Results

The identical counts across all three algorithms on this dataset reflect strong convergence on the contamination rate estimated by the IQR-based suggestion mechanism. While the counts are the same, the specific records flagged by each algorithm differ, as confirmed by the inter-model agreement scores in the heatmap discussed in Section 7.5.



Figure 7.3: Step 4 Noise Detection Results and Step 5 Data Quality Score

7.3.2 Consensus Voting Output

After collecting the individual label arrays from all three algorithms, the consensus voting mechanism was applied. A record was confirmed as a final anomaly only when at least two of the three models independently flagged it. Records flagged by only one algorithm were reclassified as normal in the

consensus output, filtering out detections that represent the bias or sensitivity of a single method rather than a genuine anomaly corroborated by multiple independent perspectives.

Consensus voting yielded **106 confirmed anomalies**, representing **17.29%** of the 614 records. The cleaned dataset produced after removing these 106 consensus-flagged records contains **508 records**, representing the version of the input data that the ensemble considers free of anomalous noise.

Metric	Value
Total Records	614
Consensus Anomalies	106
Consensus Anomaly Rate	17.29%
Cleaned Dataset Size	508 rows

Table 7.3: Consensus Voting Result

The reduction from 123 individual detections to 106 consensus detections indicates that 17 records were flagged by only one algorithm and not corroborated by the others. These borderline cases, records that sit near the edge of one model's decision boundary but are considered normal by the other two, are correctly excluded from the final anomaly list by the majority voting strategy, demonstrating the practical value of the ensemble approach in reducing false positives.

7.4 Data Quality Score

Following detection, the system computed a three-dimensional data quality score for the uploaded dataset.

Dimension	Score
Noise-Free Rate (after detection)	82.7%
Completeness (non-missing values)	97.2%
Uniqueness (non-duplicate rows)	99.8%
Overall Data Quality Score	90.6% - Excellent

Table 7.4: Data Quality Score

The overall score of 90.6%, rated as Excellent by the system, reflects a dataset that is largely clean but contains a meaningful proportion of anomalous records and a modest level of missing values. The completeness score of 97.2% accurately reflects the 149 missing values present in the original 614-record dataset, confirming that the quality scoring module correctly accounts for the missing data before imputation. The near-perfect uniqueness score of 99.8% indicates that only one duplicate record was detected and removed during preprocessing. The noise-free rate of 82.7% corresponds directly to the 508 clean records retained after consensus anomaly removal. Together these three dimensions provide a concise and immediately interpretable summary of the dataset's health that no single metric alone could convey.

7.5 Model Comparison and Agreement Analysis

The model comparison module produced both a bar chart of individual detection counts and a correlation heatmap quantifying pairwise agreement between the three algorithms.

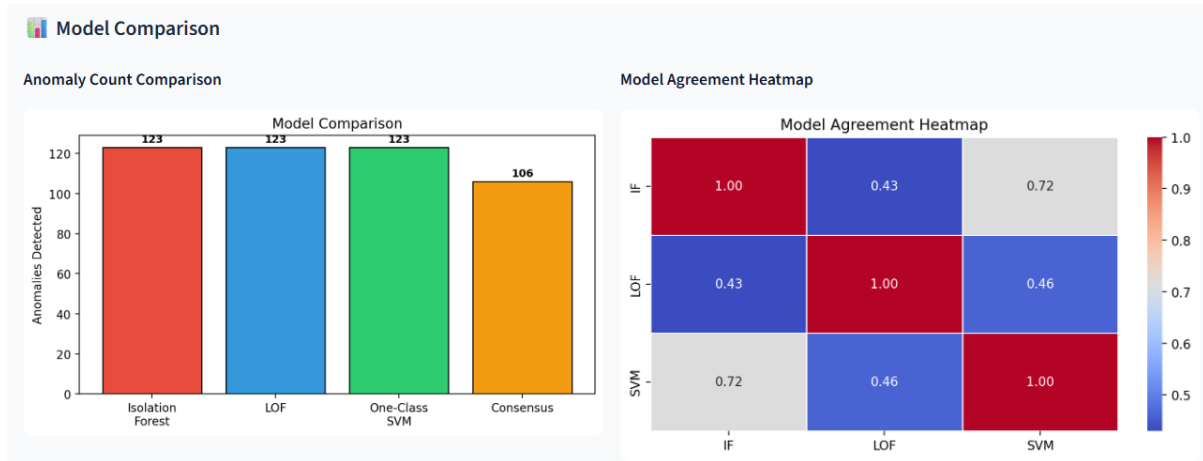


Figure 7.4: Model comparison bar chart and inter-model agreement heatmap

The bar chart confirms the equal detection counts of 123 anomalies across all three algorithms, with the consensus count of 106 shown alongside for direct comparison. The agreement heatmap reveals the pairwise Pearson correlation between each pair of model label arrays. The Isolation Forest and One-Class SVM show the strongest agreement at 0.72, indicating that these two models, one tree-based and one kernel-based, identify broadly similar sets of anomalous records on this dataset. The LOF agreement with Isolation Forest is 0.43 and with One-Class SVM is 0.46, indicating that the density-based approach identifies a meaningfully different subset of anomalies compared to the other two. This diversity in detection perspective is precisely the reason the ensemble approach is more reliable than any single algorithm: the consensus mechanism retains only the records that at least two of these different perspectives agree upon, making the final detection list more robust than what any individual method would produce alone.

7.6 Evaluation Metrics

Each model was evaluated against the consensus result as the ground truth, treating a record confirmed by the 2-of-3 majority as the positive anomaly class.

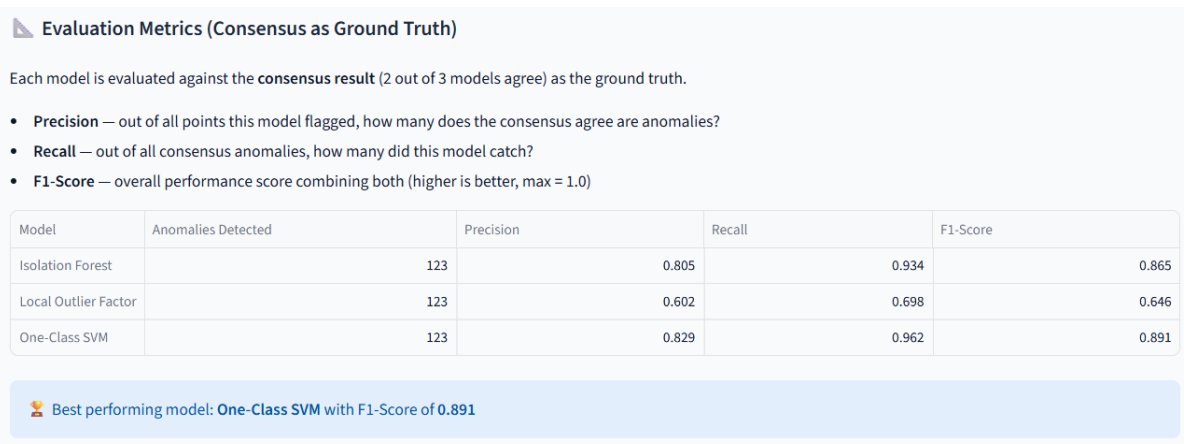


Figure 7.5: Evaluation metrics for each model against consensus ground truth

Model	Anomalies Detected	Precision	Recall	F1-Score
Isolation Forest	123	0.805	0.934	0.865
Local Outlier Factor	123	0.602	0.698	0.646
One-Class SVM	123	0.829	0.962	0.891

Table 7.5: Performance Evaluation of Individual Models Against Consensus

The One-Class SVM achieved the highest F1-Score of 0.891, with a precision of 0.829 and a recall of 0.962, making it the best-performing individual model on this dataset. This indicates that the kernel-based boundary approach is particularly well-suited to the feature distribution of the Credit Worthiness data, where the normal applicant region has a compact, learnable shape in the scaled feature space. The Isolation Forest achieved a strong F1-Score of 0.865, with a high recall of 0.934, confirming that it successfully captures the majority of true consensus anomalies. The Local Outlier Factor achieved the lowest F1-Score of 0.646, suggesting that the local density comparison approach is less aligned with the consensus definition of anomaly on this particular dataset. This variation in individual model performance further confirms the value of the consensus voting strategy: rather than relying on the LOF alone, which would miss a substantial

proportion of genuine anomalies, the ensemble combines all three perspectives so that the strengths of the Isolation Forest and One-Class SVM compensate for the LOF's lower alignment with the consensus.

7.7 Anomaly Distribution per Feature

The model comparison module produced overlapping histogram plots showing the distribution of normal and anomalous records across each selected numerical feature.

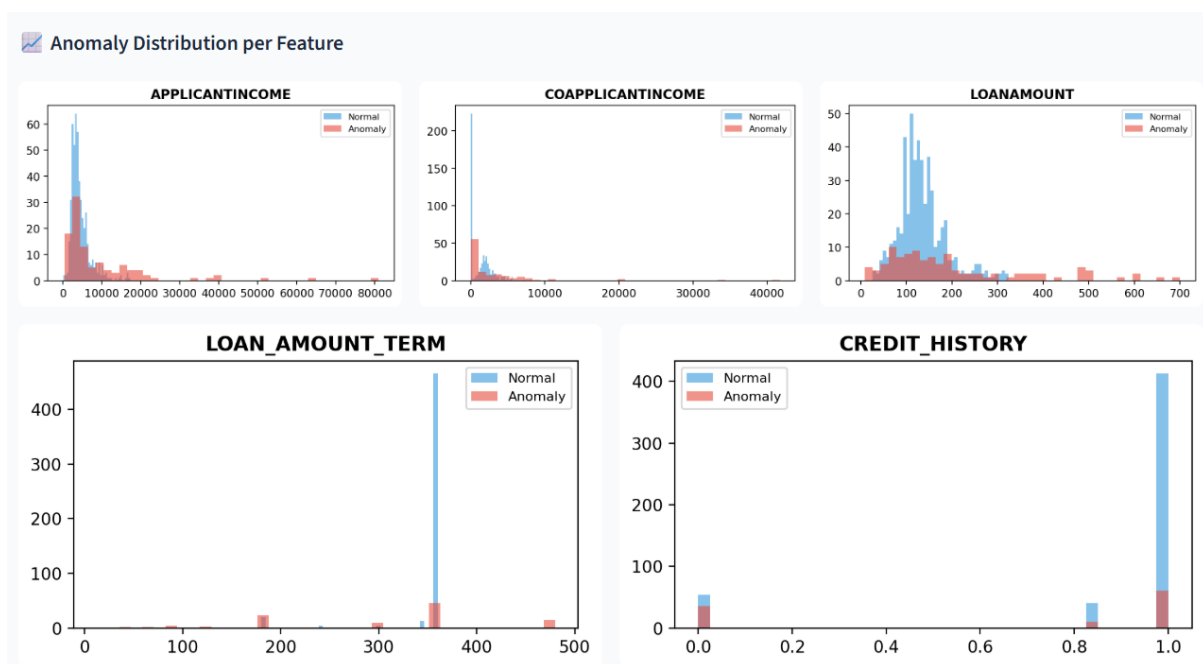


Figure 7.6: Anomaly distribution per feature- normal vs anomalous records

The distribution plots reveal several informative patterns about how anomalous records differ from normal ones across the five features. For ApplicantIncome, anomalous records are concentrated at the lower end of the income range and at a few extreme high-income values, while the normal majority clusters in the mid-range. For CoapplicantIncome, anomalous records are heavily concentrated near zero, reflecting applicants with no co-applicant income contribution, which the ensemble consistently identifies as a deviation from the typical applicant profile. For LoanAmount, anomalies are spread across a wider range than normal records, particularly at higher loan values, indicating that unusually large or unusually

small loan requests relative to income are flagged as anomalous. For Loan_Amount_Term, anomalous records are clustered at non-standard repayment periods, with the overwhelming majority of normal records concentrated at the standard 360-month term. For Credit_History, the anomaly concentration at 0.0 is striking, as records with no credit history are disproportionately flagged as anomalous, which is consistent with the financial domain understanding that the absence of credit history is a significant risk signal. These distribution patterns are consistent with the SHAP feature importance rankings discussed in the following section.

7.8 SHAP Explainability Results

SHAP values were computed for all consensus-flagged anomalies to provide feature-level explanations for every detection decision. TreeExplainer was applied for the Isolation Forest model, producing exact Shapley values efficiently by exploiting the tree structure. KernelExplainer was applied for the Local Outlier Factor and One-Class SVM models as a model-agnostic approximation. The SHAP results for each model are presented and interpreted below.

7.8.1 Isolation Forest SHAP Explanation

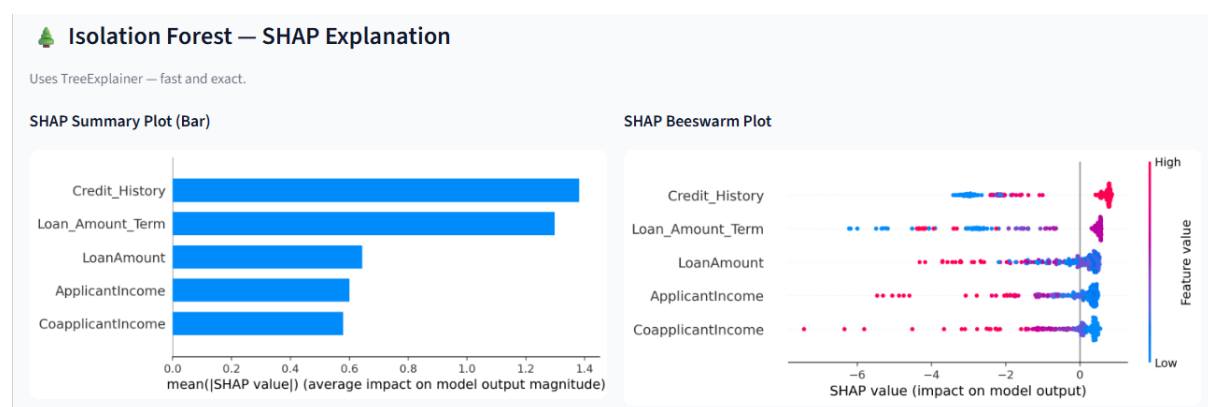


Figure 7.7: Isolation Forest SHAP summary bar chart and beeswarm plot

The SHAP summary bar chart for the Isolation Forest model ranks Credit_History as the most influential feature with a mean absolute SHAP value of approximately

1.4, followed closely by Loan_Amount_Term at approximately 1.3. LoanAmount, ApplicantIncome, and CoapplicantIncome follow with progressively smaller but still meaningful contributions. This ranking indicates that the absence of credit history and non-standard loan repayment terms are the dominant drivers of anomaly classification by the Isolation Forest on this dataset, which aligns with the distribution patterns observed in Figure 7.6.

The beeswarm plot extends this global ranking by showing the direction of each feature's influence. For Credit_History, records with low feature values corresponding to applicants with no credit history are associated with large negative SHAP values, meaning they strongly push the model's output toward the anomaly region. For Loan_Amount_Term, both low and high values produce SHAP deviations, confirming that non-standard loan terms in either direction are anomaly-driving. For LoanAmount and ApplicantIncome, high feature values are associated with positive SHAP contributions toward anomaly, indicating that unusually large loan amounts and high incomes relative to the typical applicant profile are flagged.

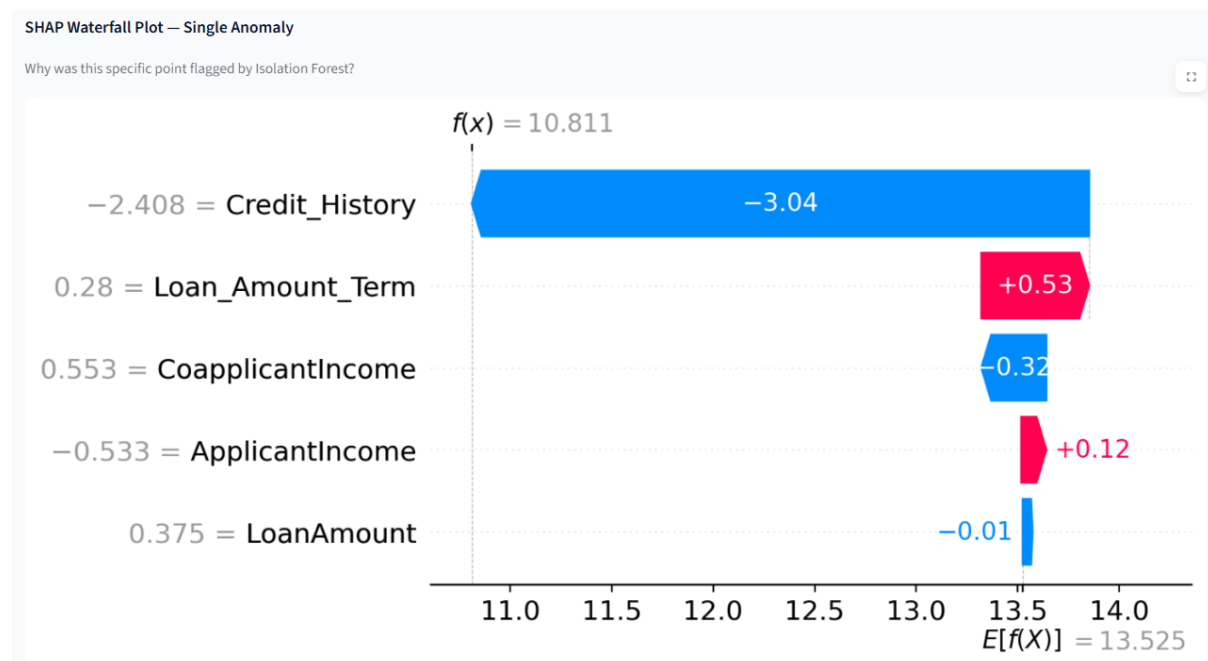


Figure 7.8: Isolation Forest SHAP waterfall chart for a single flagged anomaly

The waterfall chart provides a local, record-level explanation for a specific flagged applicant. For this record, Credit_History contributes a SHAP value of -3.04 , representing the single largest push toward the anomaly classification and reflecting that this applicant has no credit history. Loan_Amount_Term contributes $+0.53$ in the anomaly direction, while CoapplicantIncome contributes -0.32 and ApplicantIncome contributes $+0.12$ in smaller but traceable amounts. LoanAmount contributes a negligible -0.01 . The model's final output score for this record is $f(x) = 10.811$, compared to the expected baseline of $E[f(X)] = 13.525$, with the negative SHAP contributions pulling it away from the normal region. This level of per-record transparency allows a data practitioner to immediately understand that this particular applicant was flagged primarily because of the absence of a credit history, which is a directly actionable insight for downstream decision-making.

7.8.2 Local Outlier Factor SHAP Explanation

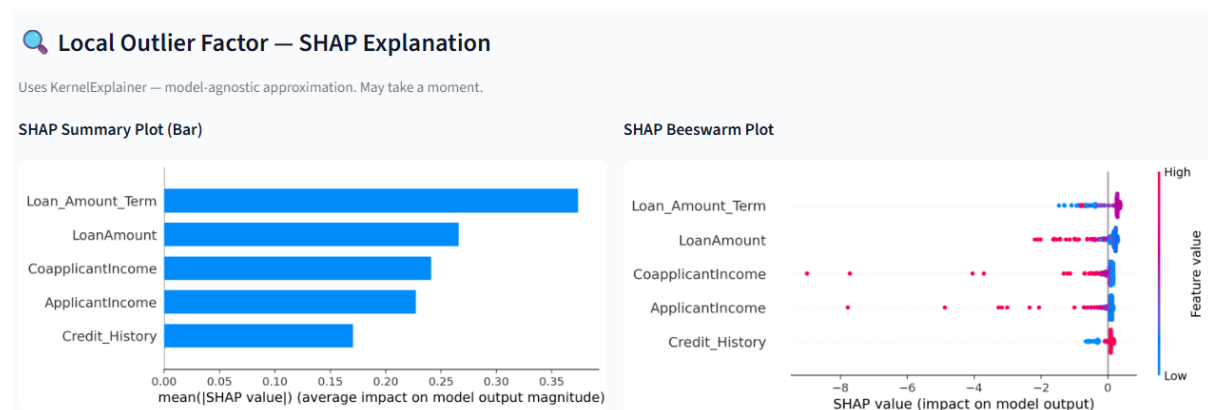


Figure 7.9: LOF SHAP summary bar chart and beeswarm plot

The SHAP summary bar chart for the Local Outlier Factor model ranks Loan_Amount_Term as the most influential feature with a mean absolute SHAP value of approximately 0.37, followed by LoanAmount at 0.26, CoapplicantIncome at 0.25, ApplicantIncome at 0.24, and Credit_History at 0.17. Compared to the Isolation Forest, the LOF assigns a more distributed importance across all five features, with no single feature dominating as strongly as

Credit_History does for the tree-based model. This reflects the density-based nature of LOF, which evaluates anomalousness relative to local neighbourhoods rather than global feature separability, meaning that the combination of financial attributes matters more than any individual signal.

The beeswarm plot confirms that for Loan_Amount_Term and LoanAmount, both low and high feature values produce negative SHAP contributions toward anomaly in the LOF model, consistent with the interpretation that records deviating from the standard loan structure in any direction are identified as locally sparse points by the density comparison.

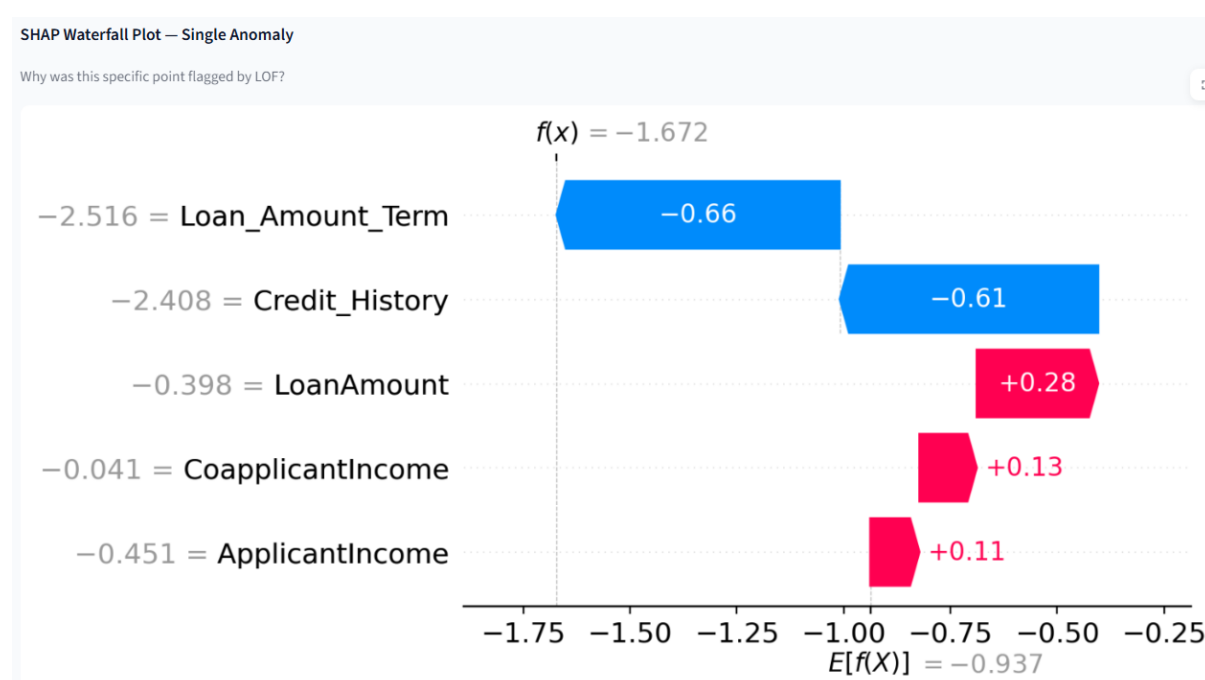


Figure 7.10: LOF SHAP waterfall chart for a single flagged anomaly

The waterfall chart for a specific LOF-flagged record shows Loan_Amount_Term contributing -0.66 and Credit_History contributing -0.61 , together accounting for the majority of the anomaly push. LoanAmount contributes $+0.28$, CoapplicantIncome $+0.13$, and ApplicantIncome $+0.11$ in smaller opposing directions. The model's output of $f(x) = -1.672$ falls well below the baseline of $E[f(X)] = -0.937$, confirming that this record lies in a region of the feature space that is substantially less dense than its nearest neighbours, and that the non-

standard loan term and absent credit history are the primary reasons for this local density deficit.

7.8.3 One-Class SVM SHAP Explanation

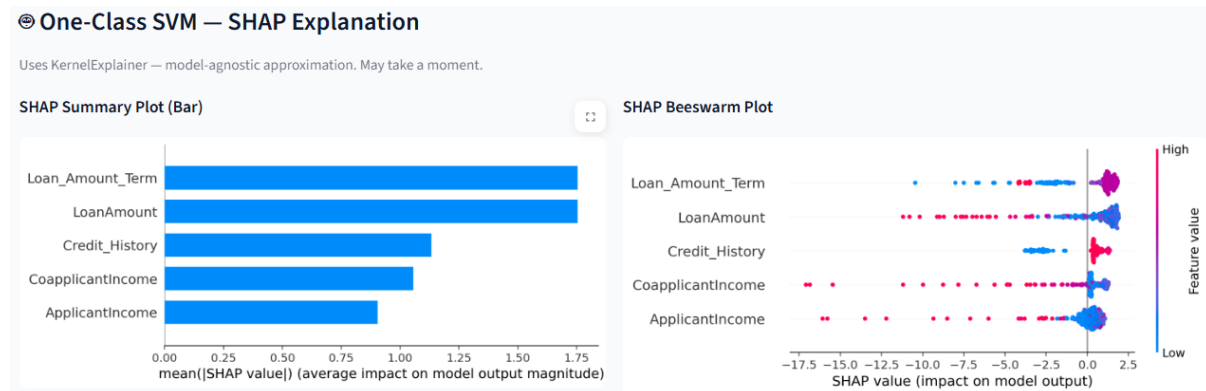


Figure 7.11: One-Class SVM SHAP summary bar chart and beeswarm plot

The SHAP summary bar chart for the One-Class SVM ranks Loan_Amount_Term and LoanAmount as jointly the most influential features, each with mean absolute SHAP values of approximately 1.75. Credit_History and CoapplicantIncome follow at approximately 1.1 each, and ApplicantIncome contributes approximately 0.85. The One-Class SVM assigns relatively high importance to the income and loan amount features compared to the Isolation Forest, reflecting the kernel-based model's sensitivity to the overall financial profile of an applicant rather than individual binary signals such as credit history presence or absence.

The beeswarm plot shows that for Loan_Amount_Term and LoanAmount, both low and high values produce large negative SHAP contributions, confirming that the One-Class SVM boundary excludes records with both unusually small and unusually large loan structures from the normal region. The widespread of SHAP values on the horizontal axis extending to - 17.5 indicates that the SVM model produces more extreme individual anomaly scores than the other two models, which is consistent with its highest recall of 0.962 observed in Table 7.5.

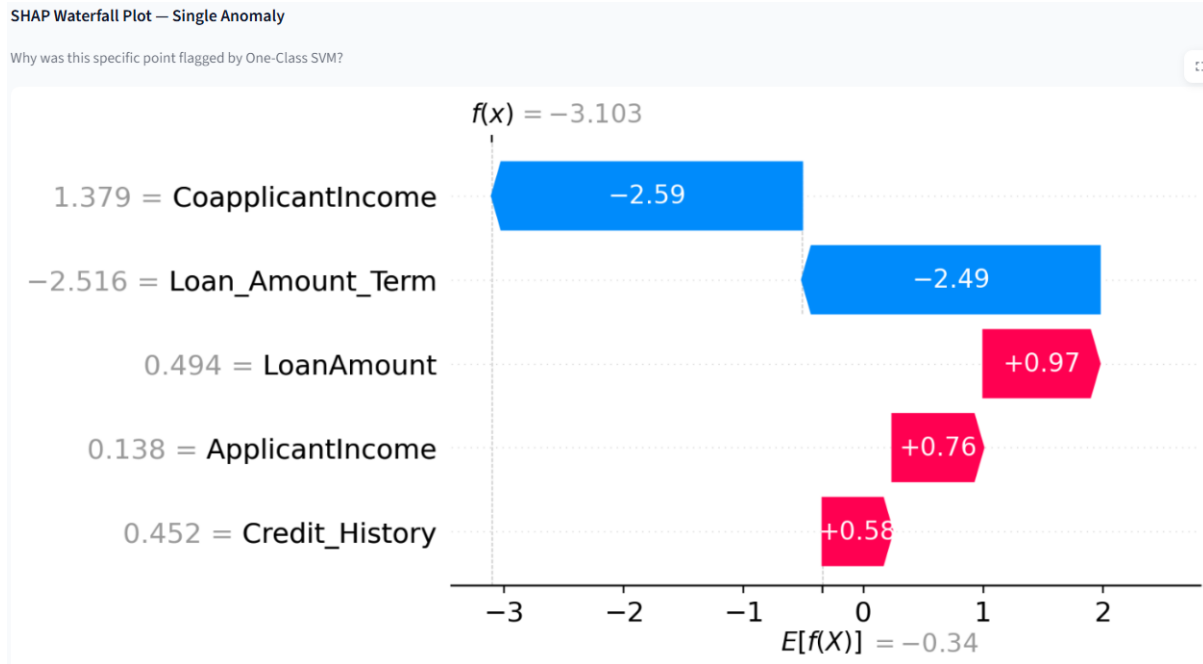


Figure 7.12: One-Class SVM SHAP waterfall chart for a single flagged anomaly

The waterfall chart for a specific One-Class SVM-flagged record shows CoapplicantIncome contributing -2.59 as the largest anomaly push, followed by Loan_Amount_Term at -2.49 . LoanAmount contributes $+0.97$ in the opposing direction, ApplicantIncome $+0.76$, and Credit_History $+0.58$. The model's output of $f(x) = -3.103$ falls substantially below the baseline of $E[f(X)] = -0.34$, indicating that this applicant lies well outside the compact normal boundary learned by the SVM. The dominant contributions from CoapplicantIncome and Loan_Amount_Term suggest that this record was flagged primarily because the co-applicant income and loan term combination places it outside the region where the majority of normal applicants cluster in the scaled feature space.

7.9 Cross-Model SHAP Comparison and Interpretive Summary

Comparing the SHAP explanations across all three models reveals both agreement and complementary perspectives on the drivers of anomaly in the Credit Worthiness dataset. Credit_History and Loan_Amount_Term are consistently identified as highly influential features across all three models,

confirming that these two attributes are the most reliable signals of anomalous applicant profiles in this dataset, regardless of which detection algorithm is used. This cross-model consistency provides strong evidence that these features represent genuine structural anomalies in the data rather than algorithm-specific artefacts. LoanAmount and income features appear with moderate and variable importance across models, reflecting that their contribution depends on the specific mathematical lens tree partitioning, density comparison, or kernel boundary, through which each model evaluates the data. The diversity in per-feature rankings across the three SHAP explanations is not a weakness; it demonstrates that each model captures a genuinely different facet of anomalous behaviour, which is precisely the justification for combining them through consensus voting rather than relying on any single perspective.

7.10 Comparison with Existing Systems

Criteria	Manual Rule-Based	Deequ	TFDV	Proposed System
Detection Method	Rule-based	Constraint-based	Statistics-based	Ensemble ML (IF + LOF + OCSVM)
Unsupervised Detection	NO	NO	Partial	YES
Ensemble Robustness	NO	NO	NO	YES (Consensus Voting)
Explainability	NO	NO	NO	YES (TreeExplainer + KernelExplainer)

Handles Missing Values	Manual	YES	YES	YES (Auto mean imputation)
Data Quality Scoring	NO	Partial	Partial	YES (Noise-free + Completeness + Uniqueness)
Auto Contamination Estimation	NO	NO	NO	YES (IQR-based)
Platform Dependency	None	Apache Spark	TensorFlow/TFX	None (Standalone Python)
Dataset Agnostic	NO	Partial	Partial	YES (Any numerical CSV, any domain)
Interactive Dashboard	NO	NO	NO	YES (Streamlit)
Open Source	YES	YES	YES	YES

Table 7.6: Comparison with Existing Systems

The comparison confirms that the proposed system is the only approach among those evaluated that simultaneously provides unsupervised ensemble detection across three algorithms, SHAP-based explainability for all three models individually, automated contamination rate suggestion, automatic missing value handling, a multi-dimensional data quality score, and an interactive dashboard. The dual-dataset evaluation across the medical and financial domains further demonstrates a dataset-agnostic capability that no existing reviewed tool can match without domain-specific configuration.

CHAPTER 8

CONCLUSION AND FUTURE WORK

This project designed, implemented, and evaluated an Explainable Artificial Intelligence-Based Automated Data Quality Management System to address two key limitations in existing numerical data quality approaches: the unreliability of single-algorithm anomaly detection and the lack of interpretability in detection outcomes. The proposed system integrates Isolation Forest, Local Outlier Factor, and One-Class SVM using a consensus voting mechanism to improve detection reliability by reducing false positives. SHAP-based explainability is incorporated through TreeExplainer and KernelExplainer to provide feature-level insights for each detected anomaly. Evaluation on the Credit Worthiness dataset demonstrates that the system effectively handles real-world data quality issues without requiring labelled data or dataset-specific configuration. Additional features such as IQR-based contamination estimation, data quality scoring, and a Streamlit dashboard enhance usability and accessibility for non-expert users.

Future work can extend the system to support categorical and mixed-type data, which would significantly broaden its applicability across diverse datasets. The inclusion of additional anomaly detection algorithms, such as Autoencoders and DBSCAN, could further improve detection accuracy and ensemble robustness. Integration with real-time data streaming platforms such as Apache Kafka would enable continuous monitoring in live data pipeline environments, while containerisation using Docker would support scalable deployment and multi-user access without requiring a local Python setup. These improvements would transform the system from a research prototype into a robust, real-world data quality management solution.

REFERENCES

- [1] Aggarwal, C. C., *Outlier Analysis*, 2nd ed. New York, USA: Springer, 2017.
- [2] Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., and Herrera, F., “Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, opportunities and Challenges,” *Information Fusion*, vol. 58, pp. 82–115, 2020.
- [3] Breck, E., Polyzotis, N., Roy, S., Whang, S. E., and Zinkevich, M., “Data Validation for Machine Learning,” in *Proc. MLSys*, 2019.
- [4] Breunig, M. M., Kriegel, H. P., Ng, R. T., and Sander, J., “LOF: Identifying Density-Based Local Outliers,” *ACM SIGMOD Record*, vol. 29, no. 2, pp. 93–104, 2000.
- [5] Chandola, V., Banerjee, A., and Kumar, V., “Anomaly Detection: A Survey,” *ACM Comput. Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [6] Cichy, V., and Rass, S., “An Overview of Data Quality Frameworks,” *IEEE Access*, vol. 7, pp. 24634–24648, 2019.
- [7] Frye, C., de Mijolla, D., Begley, T., and Goodfellow, I. J., “Shapley-Based Explainability for Anomaly Detection,” *arXiv preprint arXiv:2006.04736*, 2020.
- [8] Goldstein, M., and Uchida, S., “A Comparative Evaluation of Unsupervised Anomaly Detection Algorithms for Multivariate Data,” *PLOS ONE*, vol. 11, no. 4, 2016.
- [9] Liu, F. T., Ting, K. M., and Zhou, Z. H., “Isolation Forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [10] Lundberg, S. M., and Lee, S. I., “A Unified Approach to Interpreting Model Predictions,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.

- [11] Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., and Lee, S. I., “Consistent Individualized Feature Attribution for Tree Ensembles,” *arXiv preprint arXiv:1802.03888*, 2018.
- [12] Pipino, L. L., Lee, Y. W., and Wang, R. Y., “Data Quality Assessment,” *Commun. ACM*, vol. 45, no. 4, pp. 211–218, 2002.
- [13] Rahm, E., and Do, H. H., “Data Cleaning: Problems and Current Approaches,” *IEEE Data Eng. Bull.*, vol. 23, no. 4, pp. 3–13, 2000.
- [14] Redman, T. C., *Data Quality: The Field Guide*. Boston, USA: Digital Press, 2001.
- [15] Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., and Williamson, R. C., “Estimating the Support of a High-Dimensional Distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.